

GITFLOW LAB — 5

Git • GitHub • VS Code • Branches • Merge • Conflict • Conflict Resolution

1. Before You Start

Read one step at a time. First look at the screenshot. Then read the explanation. After that, perform the same action on your own computer.

2. Five Simple Words

Repository: Your project stored in Git.

Branch: A separate line of work.

Commit: A saved change in Git.

Push: Send your local changes to GitHub.

Merge: Bring changes from one branch into another branch.

3. Gitflow Used in This Lab

The practical uses a main branch, a develop branch, and feature branches.

```
main
├── develop
│   ├── feature/login
│   └── feature/conflict
```

The screenshots below show the complete work from creating the GitHub repository, setting up Git, creating branches, committing and pushing changes, creating a conflict, resolving it, and checking the result on GitHub.

Part A — Screenshot-Based Practical (Steps 1–58)

Each step below shows exactly what appears on screen, a simple explanation of what is happening, the Git command used (where applicable), and the corresponding screenshot from the practical session.

Step 1. Create the GitHub Repository

What you see: Open GitHub and create a new repository named Gitflow-Lab-5. In the screenshot, the repository is Public and the description is "For Task-5".

Simple explanation: This is the first step. The repository is the online place where the lab project will be stored.

Repositories contain a project's files and version history. Have a project idea? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *
Nikhilsr95 / Repository name *
gitflow lab

✓ Your new repository will be created as gitflow-lab. The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

Great repository names are short and memorable. How about [jubilant-system](#)?

Description
0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository
Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#)
Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)
No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#)
No license

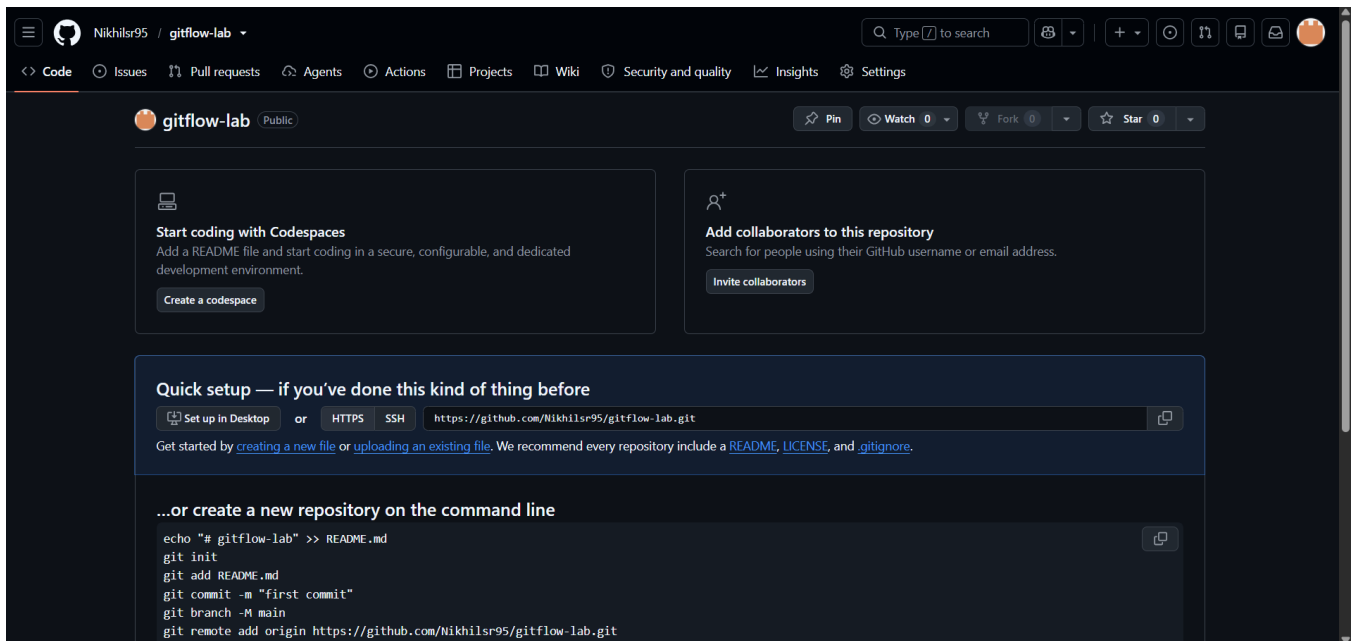
Create repository

Screenshot 1 — Step 1

Step 2. Open the New Repository

What you see: After creating the repository, GitHub opens the Gitflow-Lab-5 repository page. The HTTPS repository address is visible.

Simple explanation: This page is the home page of the GitHub repository. We will connect the local project to this repository.

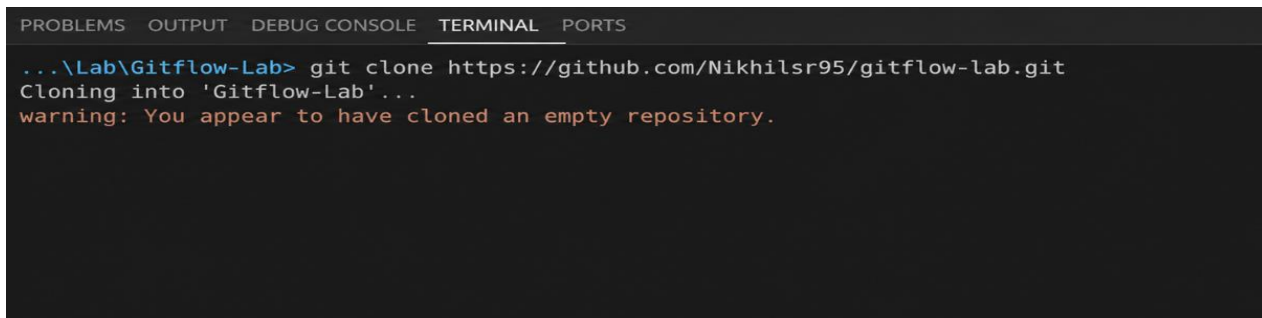


Screenshot 2 — Step 2

Step 3. Understand the Clone Command

What you see: The terminal shows a clone command and a "Repository not found" message for the example address.

Simple explanation: A clone command copies a GitHub repository to the computer. The important idea is that the URL must point to an existing repository.

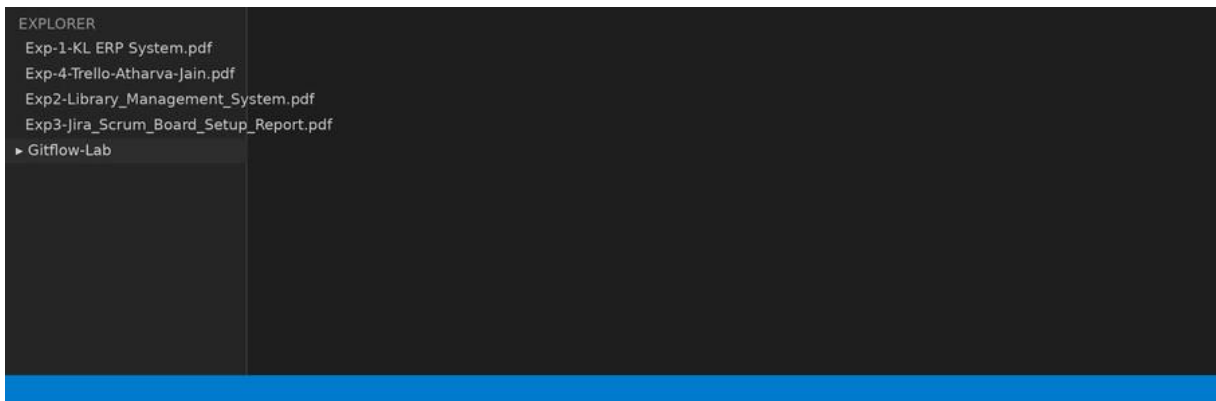


Screenshot 3 — Step 3

Step 4. Open the Project in VS Code

What you see: The VS Code Explorer shows the FSD workspace and the project folders.

Simple explanation: VS Code is used to work with the files and to run Git commands from the terminal.



Screenshot 4 — Step 4

Step 5. Move to the Project Folder

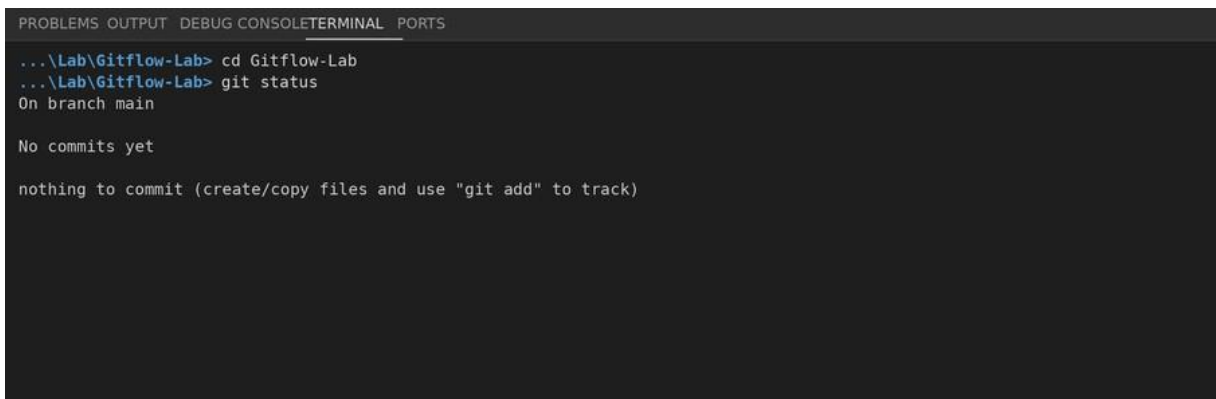
What you see: The PowerShell terminal moves into D: FSD and then into the Gitflow-Lab-5 folder.

Simple explanation: Git commands should be run inside the project folder.

Command:

```
cd Gitflow-Lab-5
```

Purpose: Moves the terminal into the project folder.



Screenshot 5 — Step 5

Step 6. Create README.md

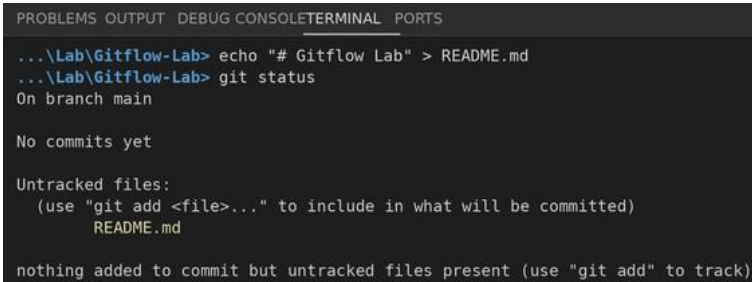
What you see: README.md is created and git status shows it as an untracked file on the main branch.

Simple explanation: README.md is a simple file used to keep information about the project. Git is now detecting it as a new file.

Command:

```
git status
```

Purpose: Checks the current branch and file status.



```
PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
...\Lab\Gitflow-Lab> echo "# Gitflow Lab" > README.md
...\Lab\Gitflow-Lab> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md

nothing added to commit but untracked files present (use "git add" to track)
```

Screenshot 6 — Step 6

Step 7. Stage README.md

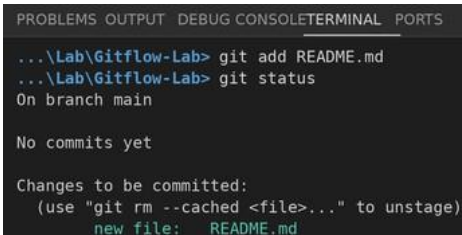
What you see: The command `git add README.md` is entered.

Simple explanation: `git add` puts the selected file into the staging area. Staging means the file is ready to be saved in a commit.

Command:

```
git add README.md
```

Purpose: Stages README.md for the next commit.



```
PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
...\Lab\Gitflow-Lab> git add README.md
...\Lab\Gitflow-Lab> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md
```

Screenshot 7 — Step 7

Step 8. Check the Staged File

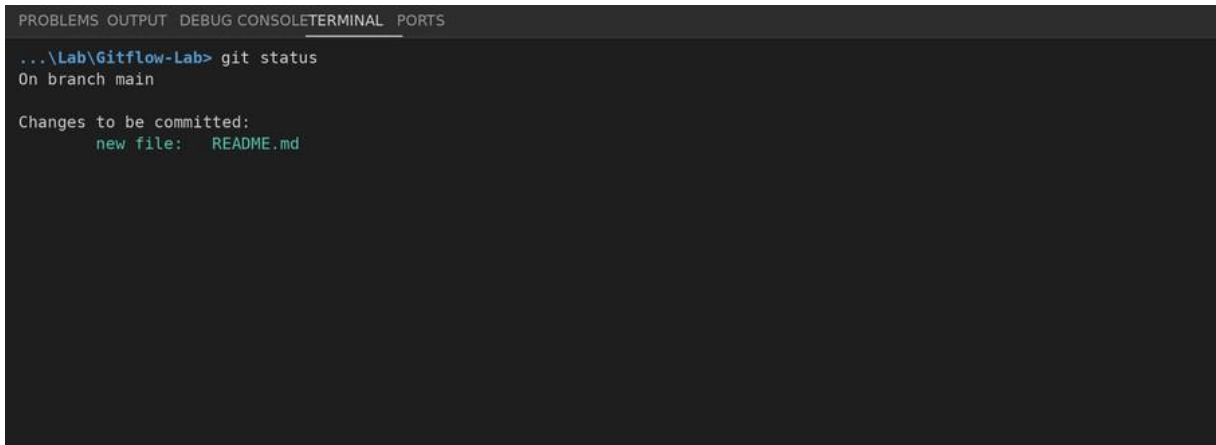
What you see: `git status` shows README.md under "Changes to be committed".

Simple explanation: This confirms that README.md has been staged successfully.

Command:

```
git status
```

Purpose: Confirms that README.md is staged.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
...\\Lab\\Gitflow-Lab> git status
On branch main

Changes to be committed:
  new file:   README.md
```

Screenshot 8 — Step 8

Step 9. Make the Initial Commit

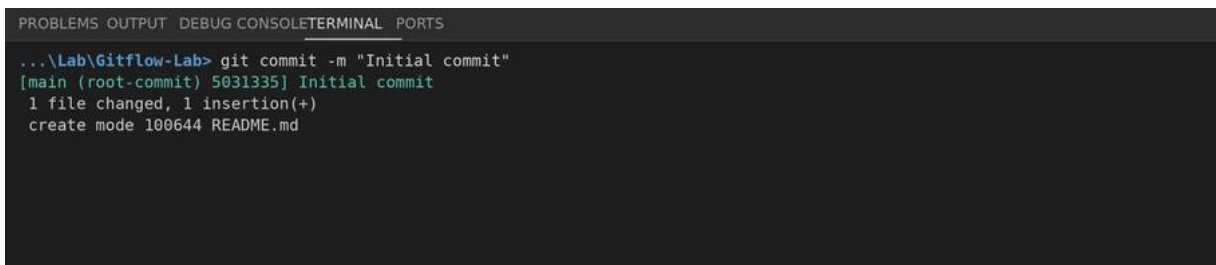
What you see: The terminal shows the initial commit being created successfully.

Simple explanation: git commit saves the staged changes in Git history. The first saved version is the initial commit.

Command:

```
git commit -m "Initial commit"
```

Purpose: Saves the first project version.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
...\\Lab\\Gitflow-Lab> git commit -m "Initial commit"
[main (root-commit) 5031335] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
```

Screenshot 9 — Step 9

Step 10. Check the Remote Command

What you see: The terminal shows Git's usage/error message after an incorrect remote command.

Simple explanation: A Git command must be written in the correct format. If Git shows usage information, check the command before continuing.

```
PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
...\Lab\Gitflow-Lab> git remote add origin https://github.com/jain-here/Gitflow-Lab.git
...\Lab\Gitflow-Lab> git remote -v
origin https://github.com/jain-here/Gitflow-Lab.git (fetch)
origin https://github.com/jain-here/Gitflow-Lab.git (push)
```

Screenshot 10 — Step 10

Step 11. Check the GitHub Remote

What you see: git remote -v shows the origin repository address.

Simple explanation: origin is the short name used for the GitHub repository connected to the local project.

Command:

```
git remote -v
```

Purpose: Shows the GitHub repository connected as origin.

```
PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
...\Lab\Gitflow-Lab> git remote -v
origin https://github.com/jain-here/Gitflow-Lab.git (fetch)
origin https://github.com/jain-here/Gitflow-Lab.git (push)
```

Screenshot 11 — Step 11

Step 12. Check the Commit History

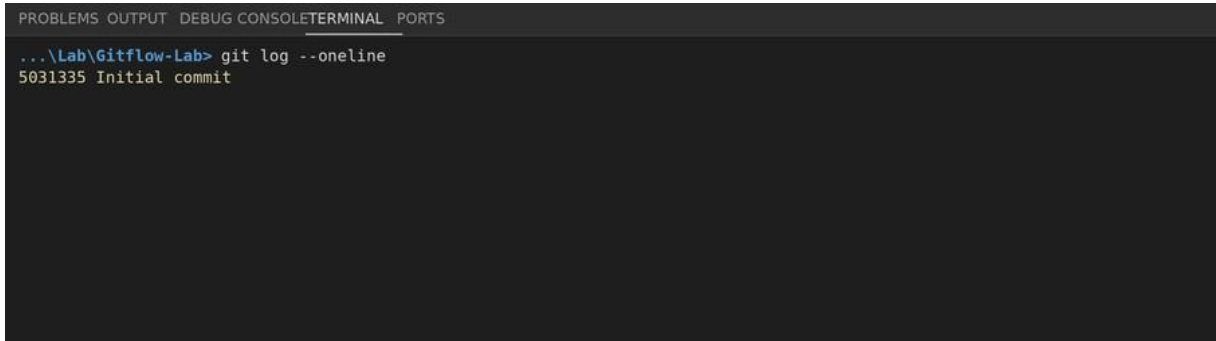
What you see: The terminal shows git log --oneline with the Initial commit.

Simple explanation: git log --oneline gives a short view of the saved commits.

Command:

```
git log --oneline
```

Purpose: Shows a short list of commits.



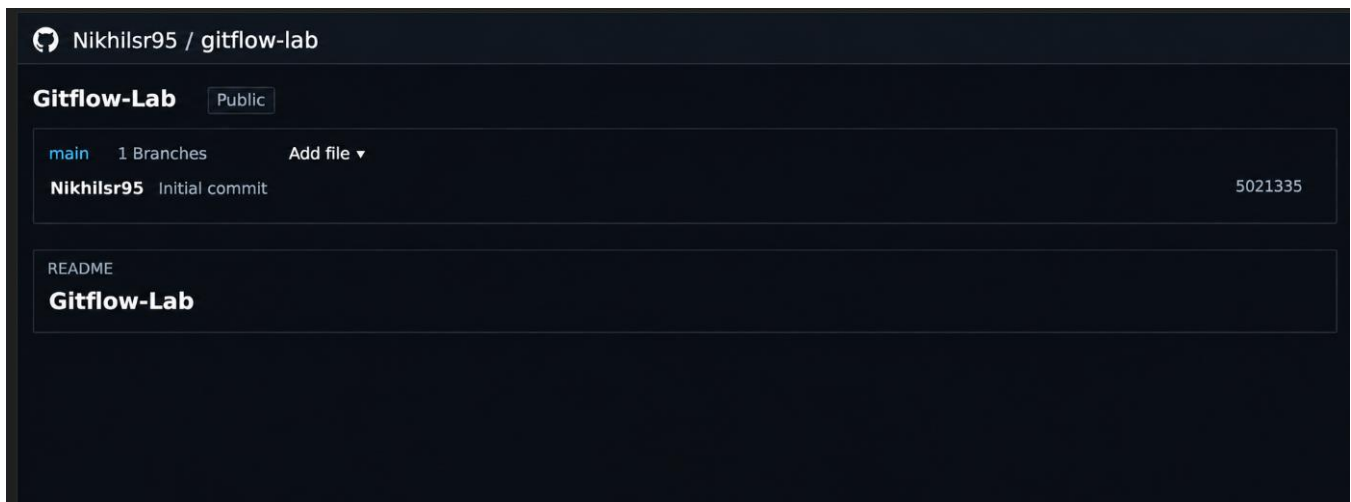
```
PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
...\\Lab\\Gitflow-Lab> git log --oneline
5031335 Initial commit
```

Screenshot 12 — Step 12

Step 13. Check the Repository on GitHub

What you see: GitHub shows the main branch, README.md, and the Initial commit.

Simple explanation: This confirms that the repository exists on GitHub and the initial project content is visible.

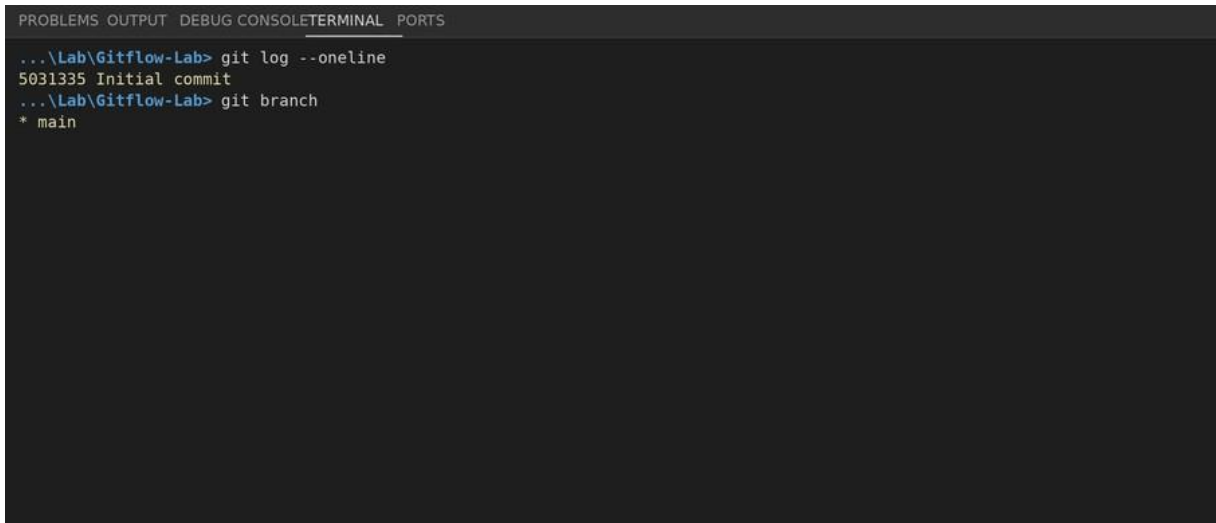


Screenshot 13 — Step 13

Step 14. Check Git Remote Carefully

What you see: The terminal again shows Git's remote command usage/error information.

Simple explanation: If the command is entered incorrectly, Git explains the correct usage. Follow the command format carefully.

A screenshot of a terminal window with a dark background. The terminal has tabs at the top labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The text in the terminal shows the following commands and output:

```
...\Lab\Gitflow-Lab> git log --oneline
5031335 Initial commit
...\Lab\Gitflow-Lab> git branch
* main
```

Screenshot 14 — Step 14

Step 15. Create the develop Branch

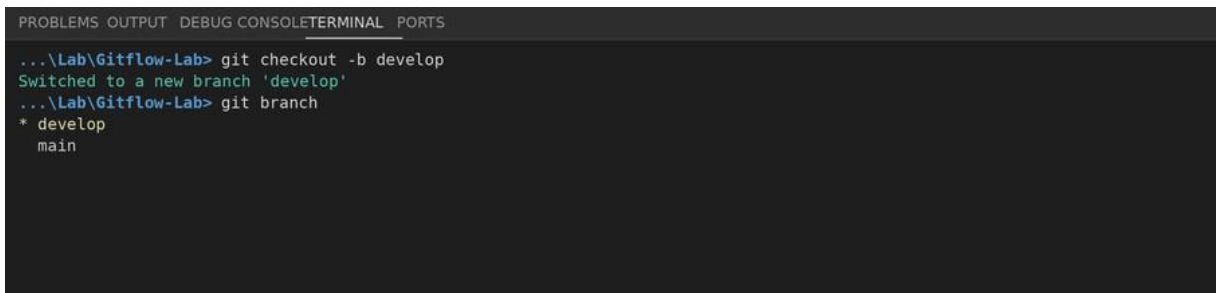
What you see: The command `git checkout -b develop` creates and switches to the develop branch. `git branch` lists develop and main.

Simple explanation: develop is the branch used for development work before changes are moved to main.

Command:

```
git checkout -b develop
```

Purpose: Creates and switches to develop.

A screenshot of a terminal window with a dark background. The terminal has tabs at the top labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The text in the terminal shows the following commands and output:

```
...\Lab\Gitflow-Lab> git checkout -b develop
Switched to a new branch 'develop'
...\Lab\Gitflow-Lab> git branch
* develop
  main
```

Screenshot 15 — Step 15

Step 16. Push develop to GitHub

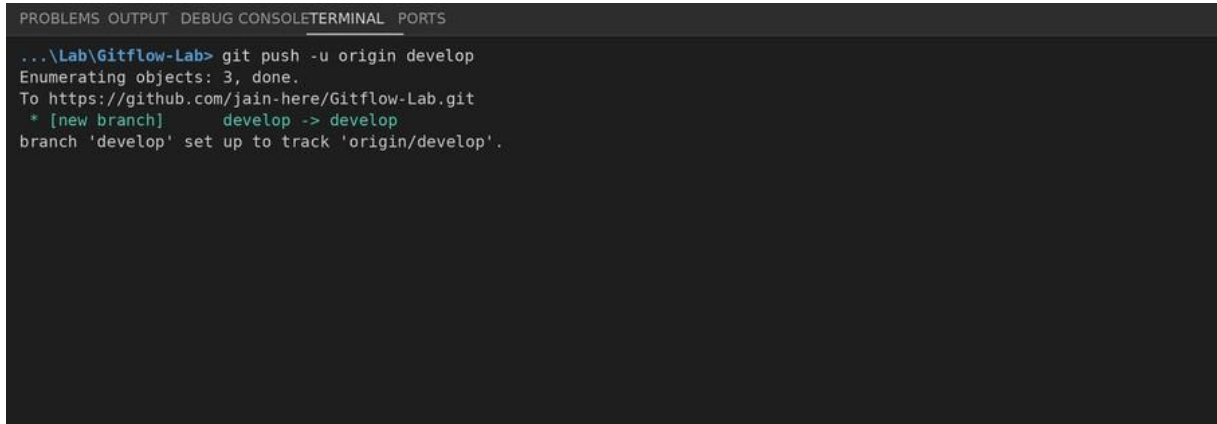
What you see: The terminal shows `git push -u origin develop` completing successfully.

Simple explanation: The `-u` option connects the local develop branch with the remote develop branch on GitHub.

Command:

```
git push -u origin develop
```

Purpose: Uploads develop to GitHub and sets its upstream.

A screenshot of a terminal window with a dark background. At the top, there is a tab bar with 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The terminal shows the following text:

```
...\\Lab\\Gitflow-Lab> git push -u origin develop
Enumerating objects: 3, done.
To https://github.com/jain-here/Gitflow-Lab.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
```

Screenshot 16 — Step 16

Step 17. Create feature/login

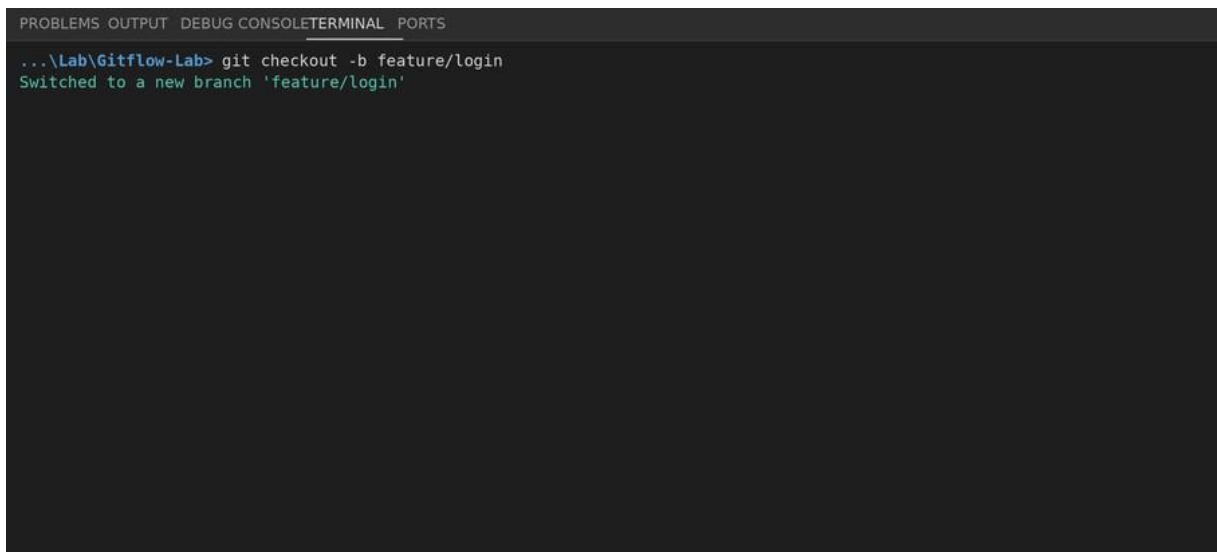
What you see: The terminal switches from develop to a new feature/login branch.

Simple explanation: A feature branch is used for one particular feature. Here the feature is login.

Command:

```
git checkout -b feature/login
```

Purpose: Creates and switches to feature/login.

A screenshot of a terminal window with a dark background. At the top, there is a tab bar with 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The terminal shows the following text:

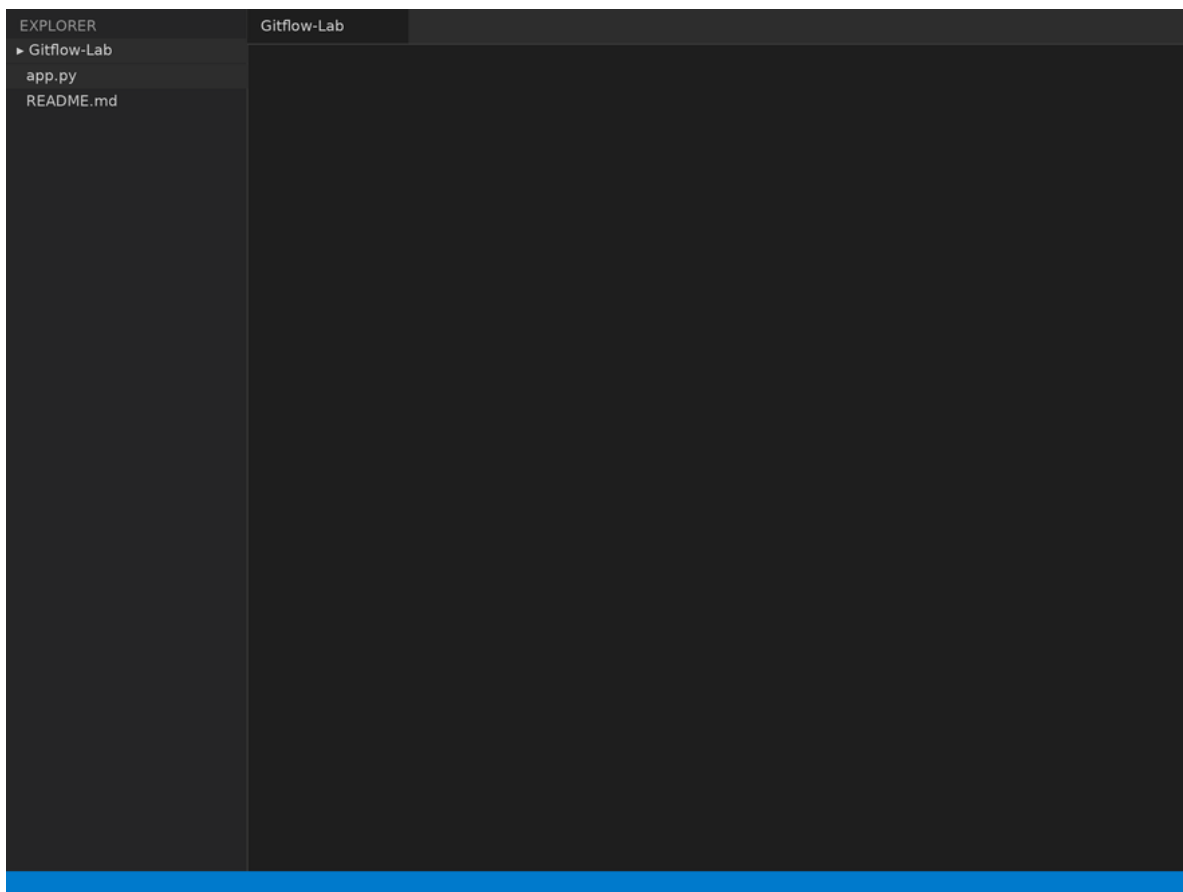
```
...\\Lab\\Gitflow-Lab> git checkout -b feature/login
Switched to a new branch 'feature/login'
```

Screenshot 17 — Step 17

Step 18. Create app.py

What you see: The VS Code Explorer shows app.py as a new file in Gitflow-Lab-5.

Simple explanation: app.py will contain the small Python program used in this lab.

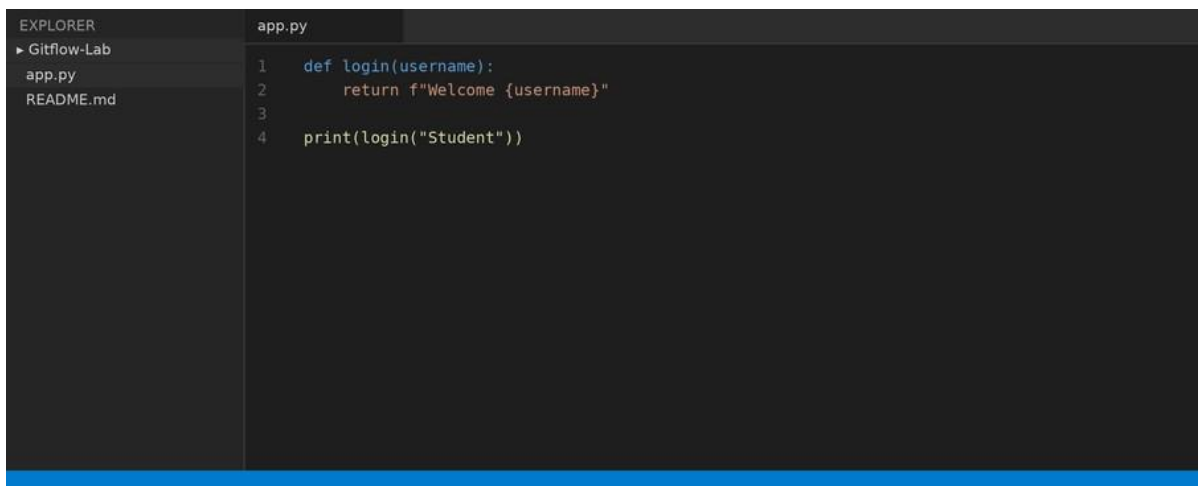


Screenshot 18 — Step 18

Step 19. Write the Login Program

What you see: The editor shows the login function and `print(login("Student"))` in app.py.

Simple explanation: The program receives a username and returns a welcome message.

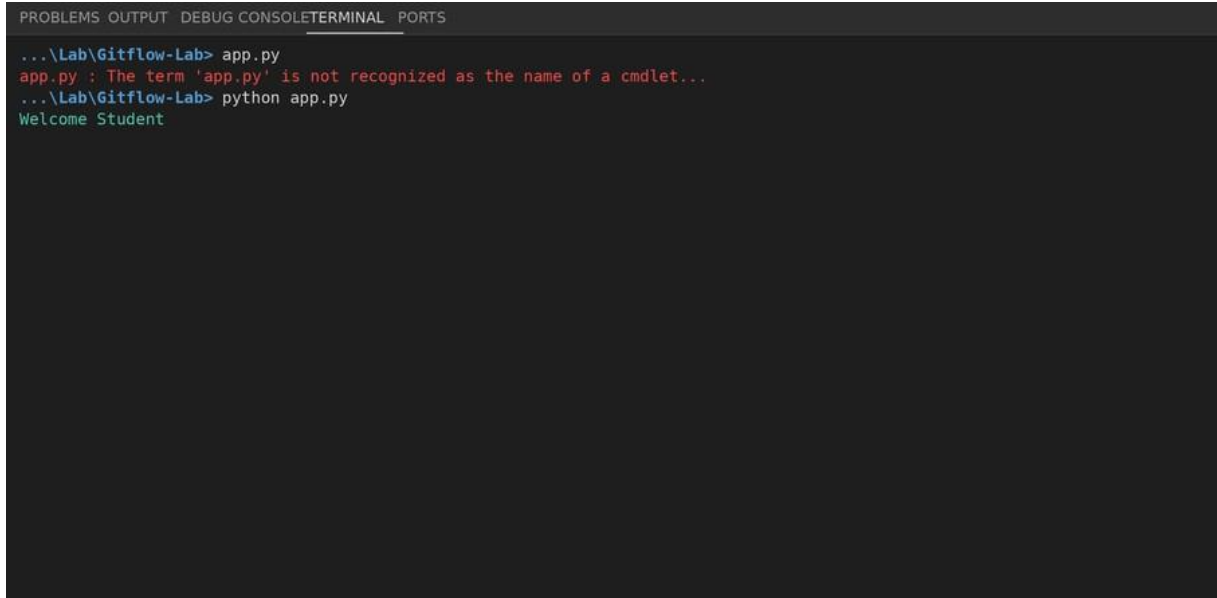


Screenshot 19 — Step 19

Step 20. Check the Python Command

What you see: The terminal shows an error after `app.py` was entered directly as a PowerShell command.

Simple explanation: A Python file should be run using Python, for example: `python app.py`.

A screenshot of a Visual Studio Code terminal window. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, and TERMINAL. The TERMINAL tab is active. The prompt is `...\Lab\Gitflow-Lab>`. The user enters `app.py`, and the terminal shows an error: `app.py : The term 'app.py' is not recognized as the name of a cmdlet...`. The user then enters `python app.py`, and the terminal shows the output: `Welcome Student`.

Screenshot 20 — Step 20

Step 21. Run `app.py`

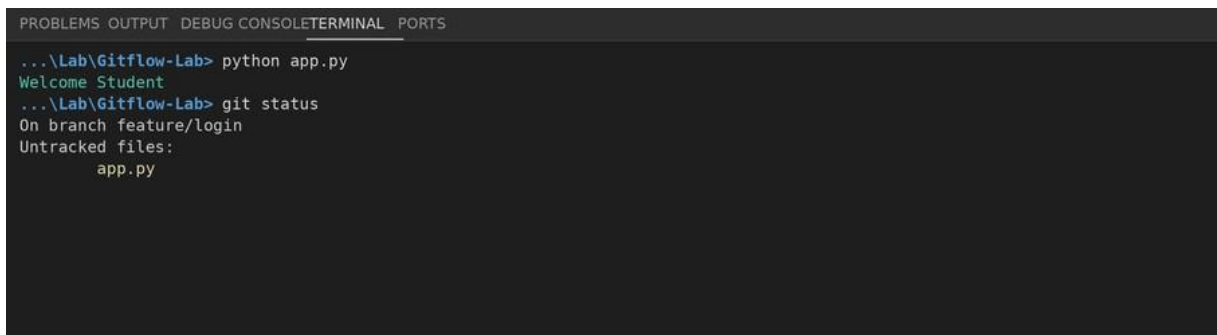
What you see: The terminal shows `python app.py` producing "Welcome Student", followed by `git status` showing `app.py` as untracked.

Simple explanation: The program is working. Git also knows that `app.py` is a new file that has not yet been staged.

Command:

```
python app.py
```

Purpose: Runs the Python program.

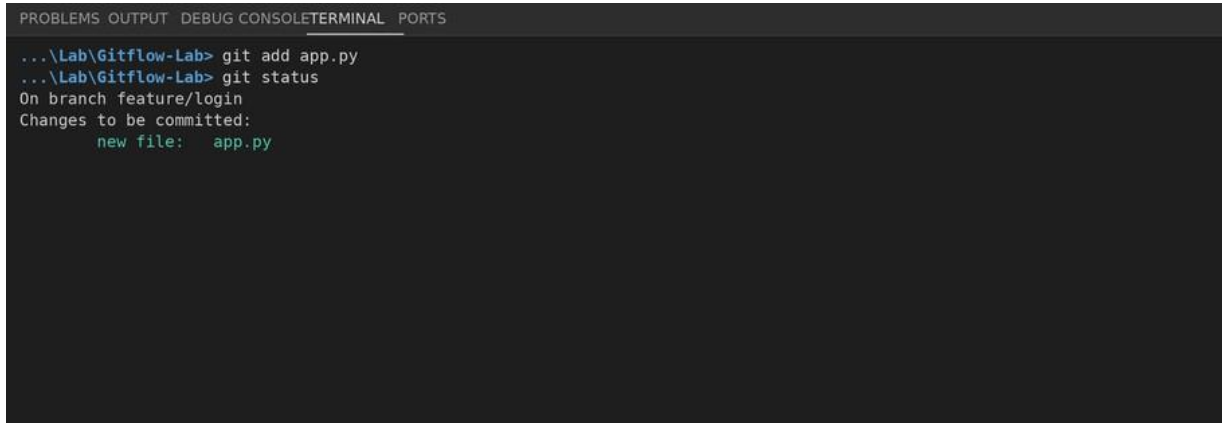
A screenshot of a Visual Studio Code terminal window. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, and TERMINAL. The TERMINAL tab is active. The prompt is `...\Lab\Gitflow-Lab>`. The user enters `python app.py`, and the terminal shows the output: `Welcome Student`. The user then enters `git status`, and the terminal shows the output: `On branch feature/login` and `Untracked files:` followed by `app.py`.

Screenshot 21 — Step 21

Step 22. Check the New File Again

What you see: The terminal again shows python app.py producing "Welcome Student" and git status showing app.py as untracked.

Simple explanation: This confirms both the program output and the Git status.

A screenshot of a terminal window with a dark background. The terminal shows the following commands and output:

```
...\\Lab\\Gitflow-Lab> git add app.py
...\\Lab\\Gitflow-Lab> git status
On branch feature/login
Changes to be committed:
  new file:   app.py
```

Screenshot 22 — Step 22

Step 23. Commit the Login Feature

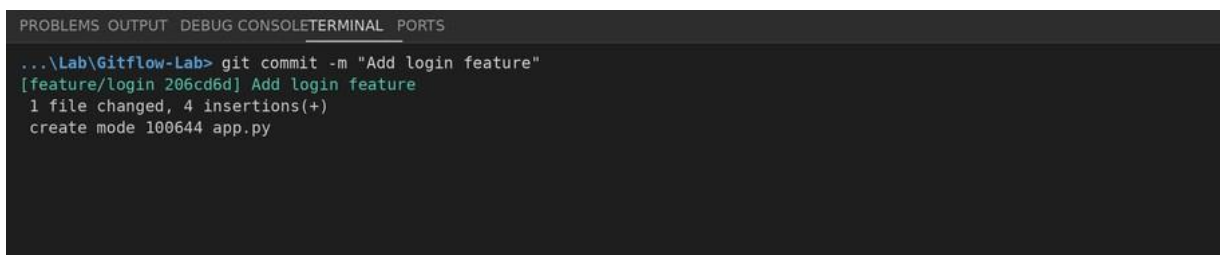
What you see: The command git commit -m "Add login feature" creates the feature/login commit.

Simple explanation: The commit message tells us what was added. This saves the login feature in Git history.

Command:

```
git commit -m "Add login feature"
```

Purpose: Saves the login feature in Git history.

A screenshot of a terminal window with a dark background. The terminal shows the following command and output:

```
...\\Lab\\Gitflow-Lab> git commit -m "Add login feature"
[feature/login 206cd6d] Add login feature
1 file changed, 4 insertions(+)
create mode 100644 app.py
```

Screenshot 23 — Step 23

Step 24. Stage app.py

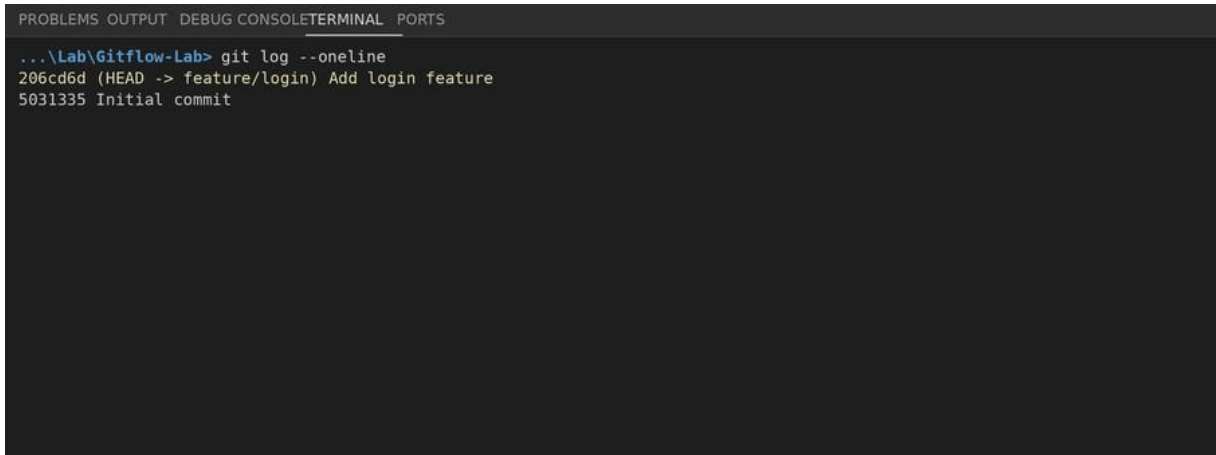
What you see: The terminal shows git add app.py and git status with app.py staged.

Simple explanation: The file is now ready to be included in the next commit.

Command:

```
git add app.py
```

Purpose: Stages app.py.

A screenshot of a terminal window with a dark background. The terminal title bar shows 'PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS'. The command prompt is '...\Lab\Gitflow-Lab>'. The user has entered 'git log --oneline'. The output shows two commits: '206cd6d (HEAD -> feature/login) Add login feature' and '5031335 Initial commit'.

Screenshot 24 — Step 24

Step 25. Check and Push feature/login

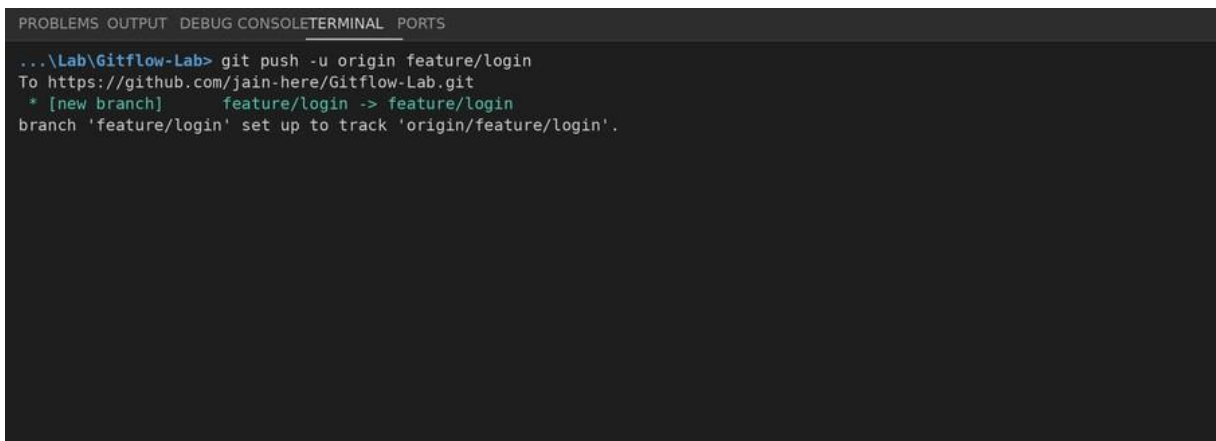
What you see: The terminal shows the Add login feature commit, the short log, and the feature/login push.

Simple explanation: This step checks the feature history and sends the feature branch to GitHub.

Command:

```
git push -u origin feature/login
```

Purpose: Uploads feature/login to GitHub.

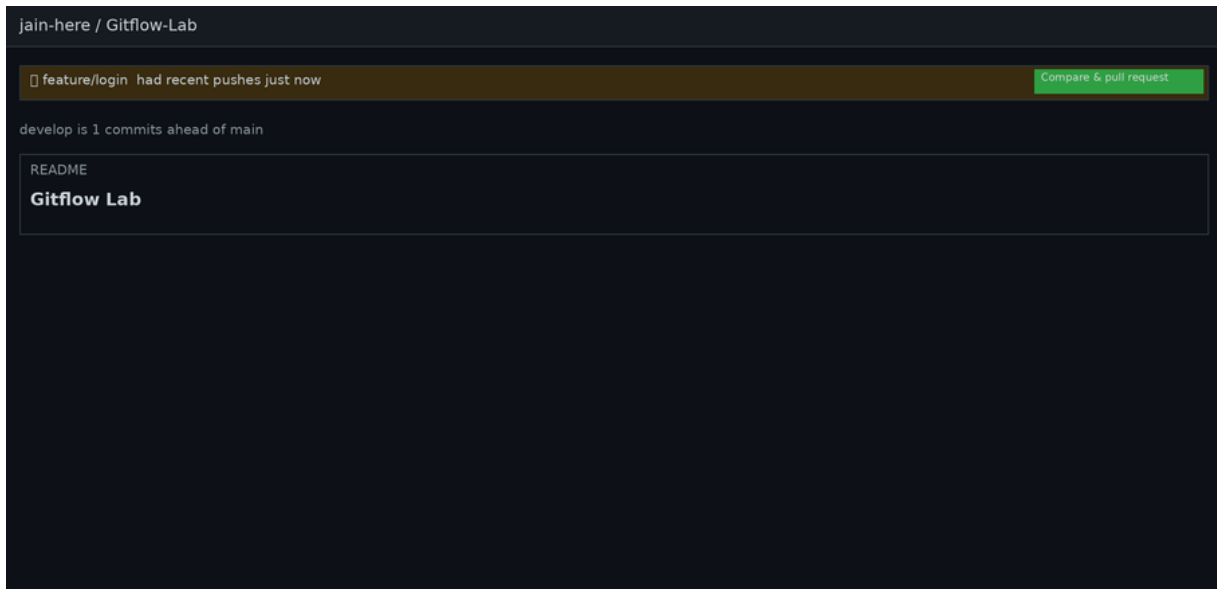
A screenshot of a terminal window with a dark background. The terminal title bar shows 'PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS'. The command prompt is '...\Lab\Gitflow-Lab>'. The user has entered 'git push -u origin feature/login'. The output shows the push to 'https://github.com/jain-here/Gitflow-Lab.git', indicating a new branch 'feature/login' is being pushed and set up to track 'origin/feature/login'.

Screenshot 25 — Step 25

Step 26. See feature/login on GitHub

What you see: The GitHub repository page shows a recent push for feature/login.

Simple explanation: This confirms that the feature branch has been uploaded to GitHub.

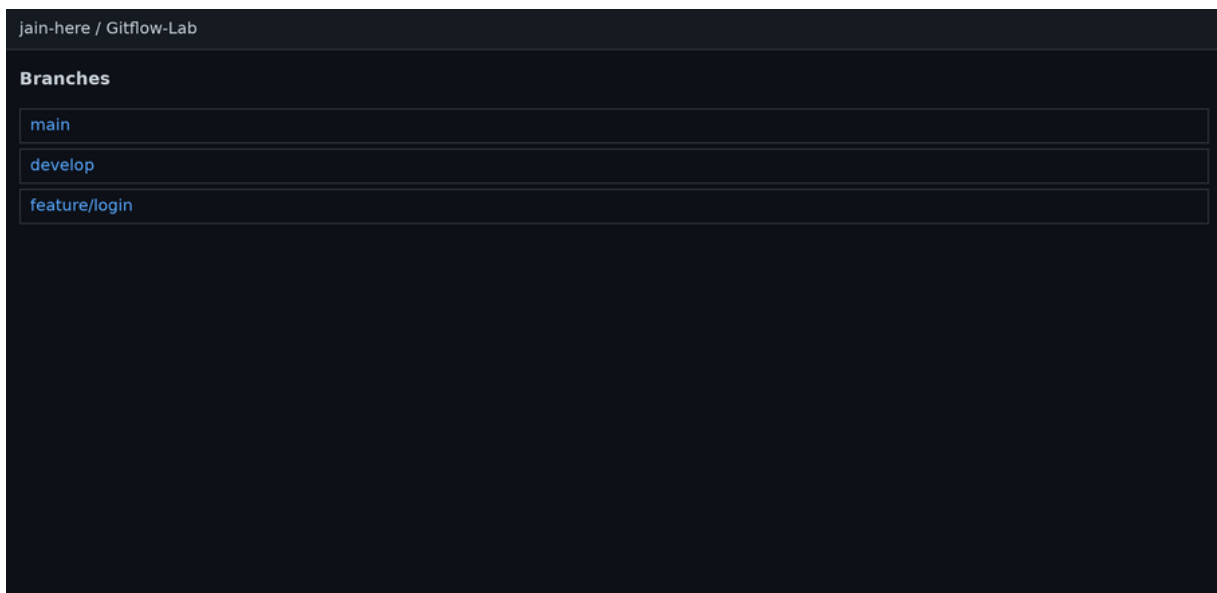


Screenshot 26 — Step 26

Step 27. Open feature/login Branch

What you see: GitHub shows feature/login with app.py and README.md and indicates that the branch is ahead of main.

Simple explanation: The feature branch contains the login work separately from main.



Screenshot 27 — Step 27

Step 28. Review feature/login History

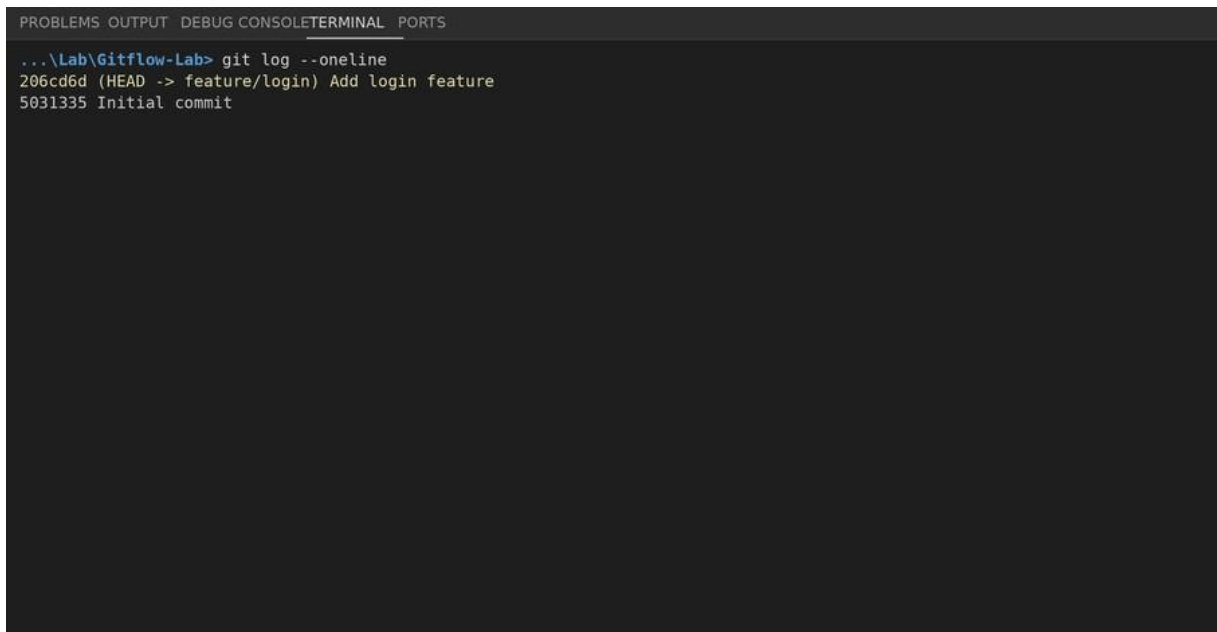
What you see: The terminal shows the staged app.py, the Add login feature commit, and the Git history.

Simple explanation: This is a quick check that the feature work was saved correctly.

Command:

```
git log --oneline
```

Purpose: Shows the short commit history.



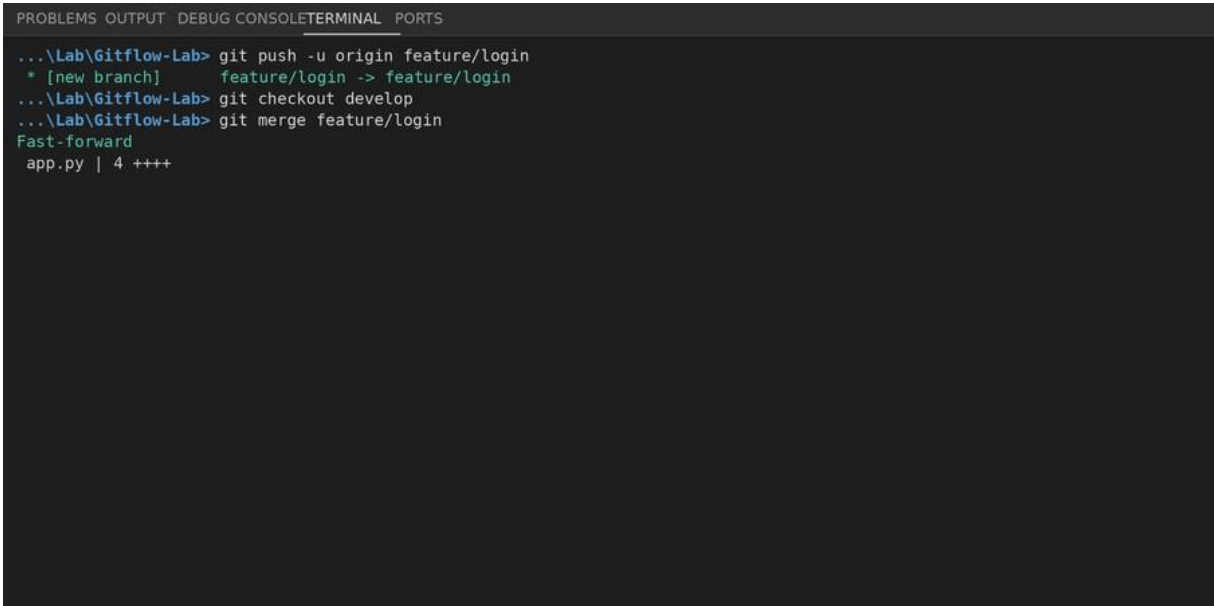
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
...\\Lab\\Gitflow-Lab> git log --oneline
206cd6d (HEAD -> feature/login) Add login feature
5031335 Initial commit
```

Screenshot 28 — Step 28

Step 29. Finish the feature/login Push

What you see: The terminal shows the push progress and successful upload to GitHub.

Simple explanation: The remote feature/login branch is now updated.

A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The terminal shows the following commands and output:

```
...\\Lab\\Gitflow-Lab> git push -u origin feature/login
* [new branch]      feature/login -> feature/login
...\\Lab\\Gitflow-Lab> git checkout develop
...\\Lab\\Gitflow-Lab> git merge feature/login
Fast-forward
app.py | 4 +++++
```

Screenshot 29 — Step 29

Step 30. Merge feature/login into develop

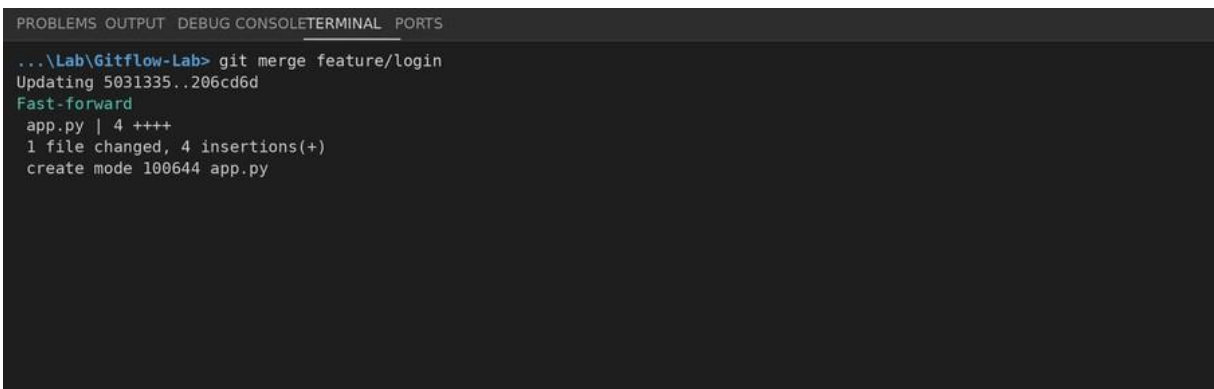
What you see: The terminal shows git merge feature/login on develop and the message Fast-forward.

Simple explanation: The login feature is now included in develop. Fast-forward means Git could move develop forward without creating a separate merge commit.

Command:

```
git merge feature/login
```

Purpose: Combines feature/login into the current develop branch.

A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The terminal shows the following commands and output:

```
...\\Lab\\Gitflow-Lab> git merge feature/login
Updating 5031335..206cd6d
Fast-forward
app.py | 4 +++++
1 file changed, 4 insertions(+)
create mode 100644 app.py
```

Screenshot 30 — Step 30

Step 31. Push Updated develop

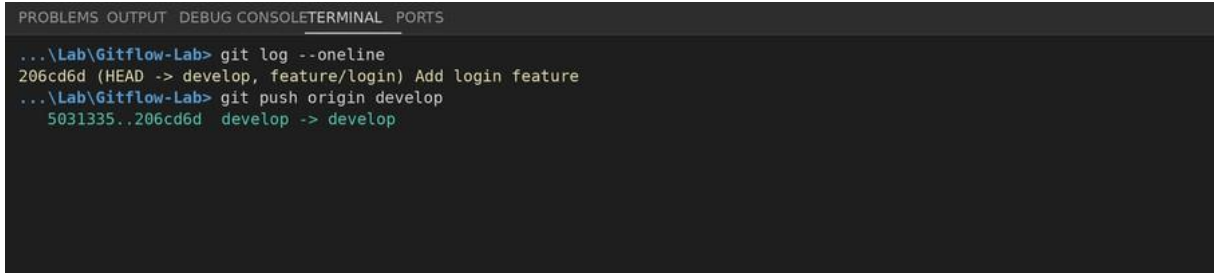
What you see: The terminal shows git log --oneline and git push origin develop completing successfully.

Simple explanation: The updated develop branch is now available on GitHub.

Command:

```
git push origin develop
```

Purpose: Uploads the updated develop branch.



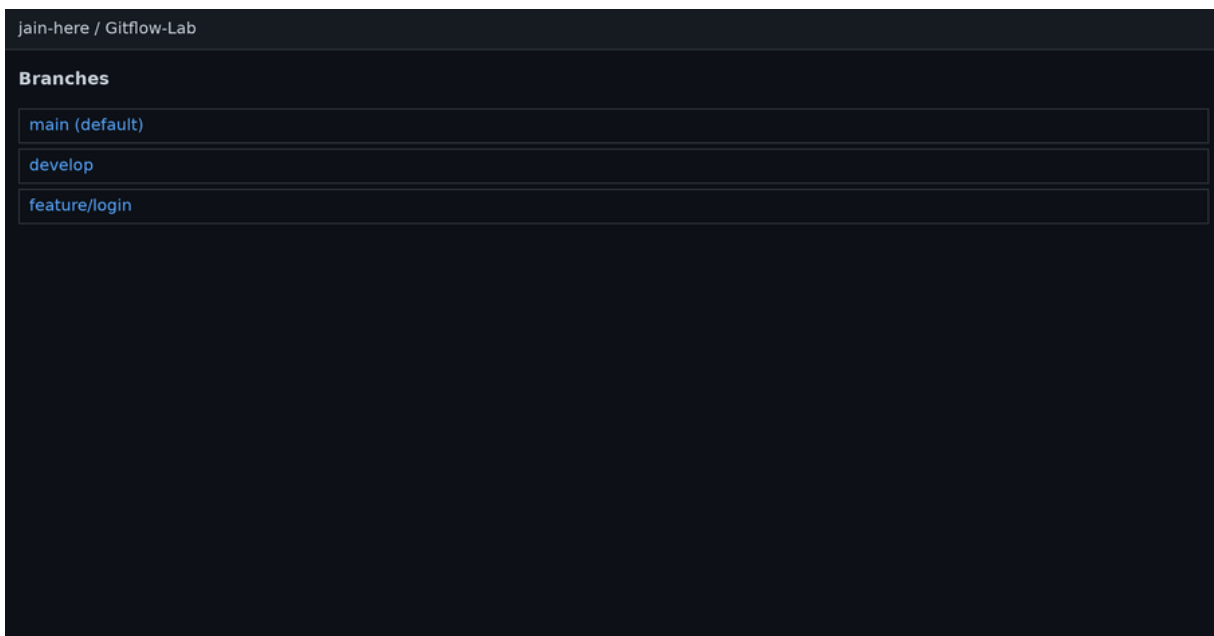
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
...\\Lab\\Gitflow-Lab> git log --oneline
206cd6d (HEAD -> develop, feature/login) Add login feature
...\\Lab\\Gitflow-Lab> git push origin develop
5031335..206cd6d develop -> develop
```

Screenshot 31 — Step 31

Step 32. Check GitHub Branches

What you see: GitHub shows main, develop, and feature/login in the branch list.

Simple explanation: This is the Gitflow branch structure created so far.



Screenshot 32 — Step 32

Step 33. Prepare the Conflict Branch

What you see: The terminal shows branch history, a develop push, the branch list, and creation of feature/conflict.

Simple explanation: feature/conflict will be used to demonstrate a merge conflict.

Command:

```
git checkout -b feature/conflict
```

Purpose: Creates and switches to feature/conflict.



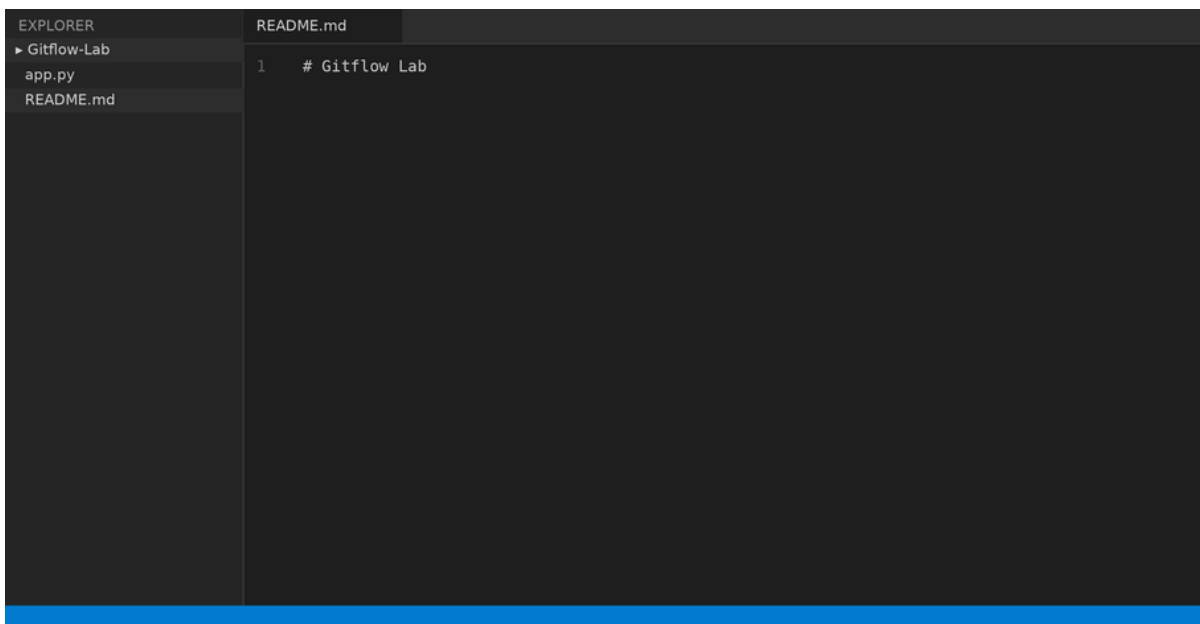
```
PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
...\Lab\Gitflow-Lab> git checkout -b feature/conflict
Switched to a new branch 'feature/conflict'
```

Screenshot 33 — Step 33

Step 34. Open README.md

What you see: VS Code shows README.md with the Gitflow Lab 5 heading while working on the conflict branch.

Simple explanation: README.md will be changed on different branches so Git has two different versions to compare.

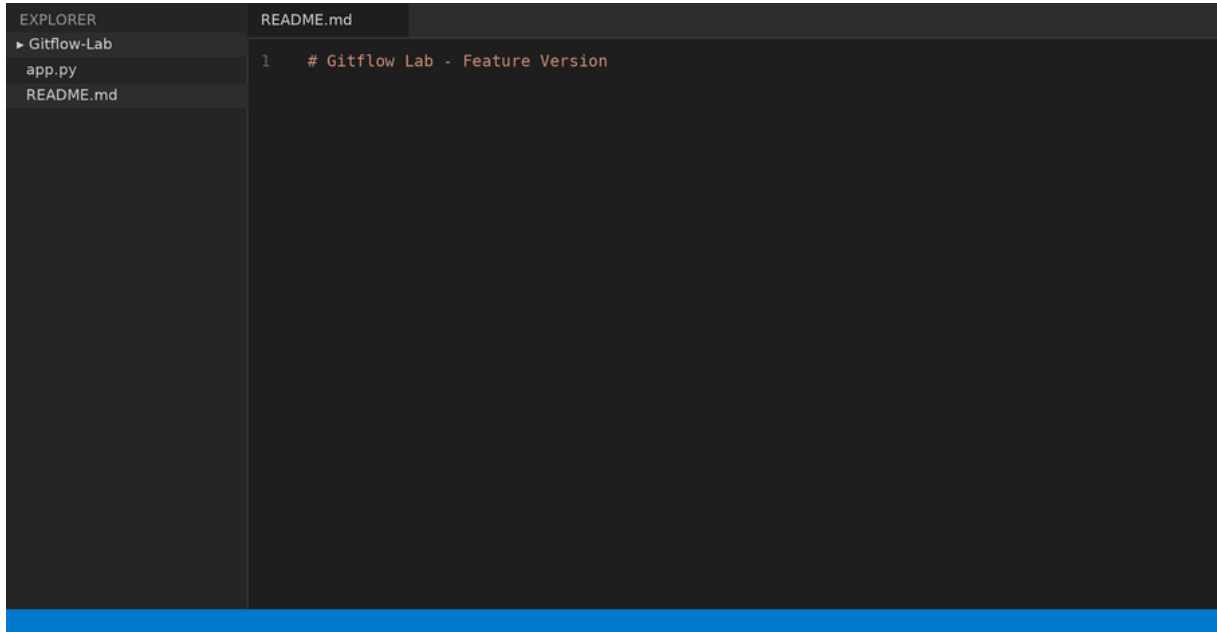


Screenshot 34 — Step 34

Step 35. Change the Feature Version

What you see: README.md shows "Gitflow Lab 5 - Feature Version".

Simple explanation: This creates the feature-side version of the README file.

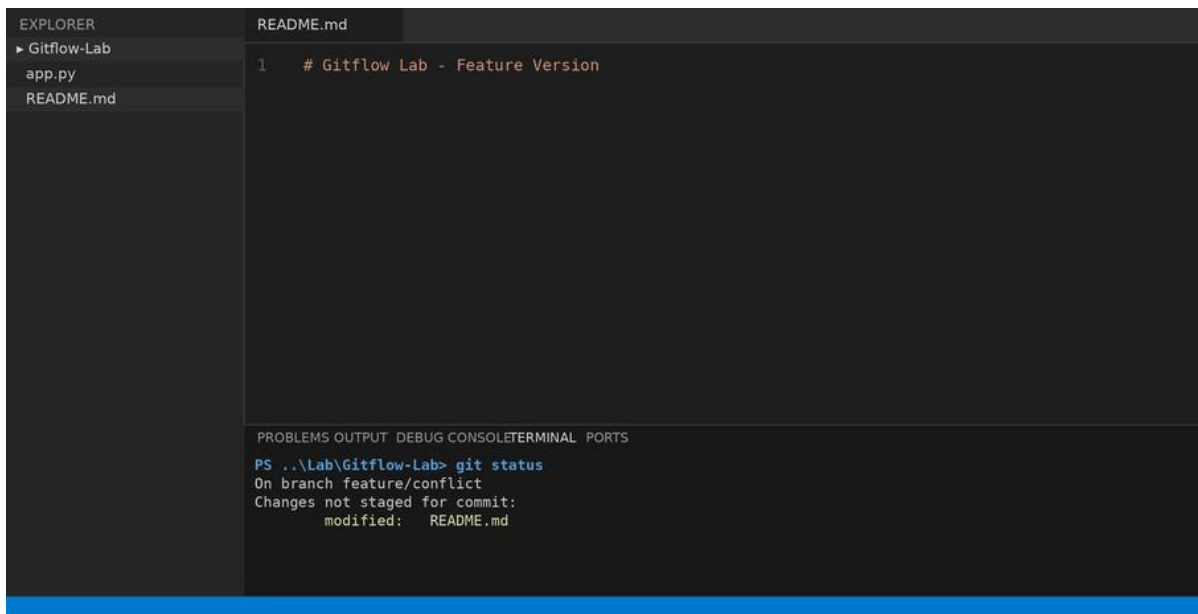


Screenshot 35 — Step 35

Step 36. Switch to feature/conflict

What you see: The terminal shows git checkout -b feature/conflict and confirms the branch switch.

Simple explanation: The conflict work is being done on this feature branch.



Screenshot 36 — Step 36

Step 37. Stage README.md on feature/conflict

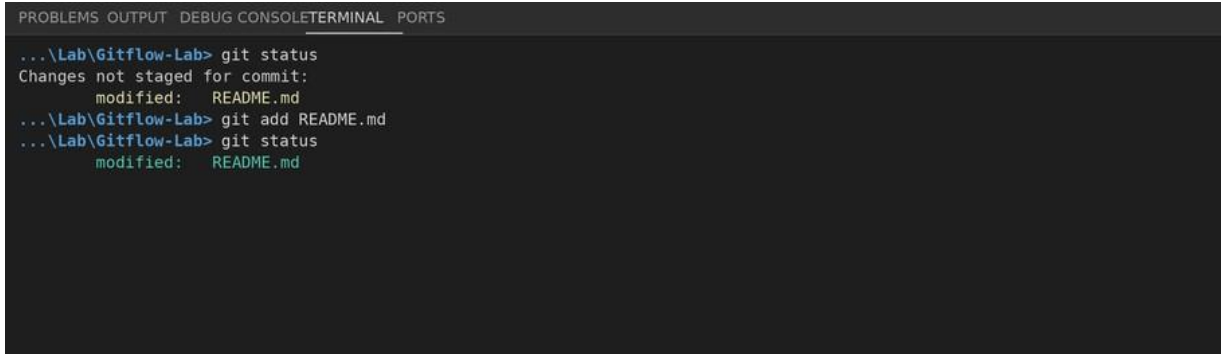
What you see: git add README.md is used and git status shows README.md staged.

Simple explanation: The feature-side README change is ready to commit.

Command:

```
git add README.md
```

Purpose: Stages the feature-side README change.



```
PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
...\\Lab\\Gitflow-Lab> git status
Changes not staged for commit:
  modified:   README.md
...\\Lab\\Gitflow-Lab> git add README.md
...\\Lab\\Gitflow-Lab> git status
  modified:   README.md
```

Screenshot 37 — Step 37

Step 38. Commit the Conflict Feature

What you see: The terminal shows the commit command for the conflict feature.

Simple explanation: The feature-side README version is saved in Git history.

Command:

```
git commit -m "Update README for conflict feature"
```

Purpose: Saves the feature-side README change.



```
PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
...\\Lab\\Gitflow-Lab> git commit -m "Update README for conflict feature"
[feature/conflict dbel5cf] Update README for conflict feature
1 file changed, 1 insertion(+), 1 deletion(-)
```

Screenshot 38 — Step 38

Step 39. Confirm the Conflict Feature Commit

What you see: The terminal shows the successful "Update README for conflict feature" commit.

Simple explanation: The feature/conflict branch now has its own README change.

```

PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
...\\Lab\\Gitflow-Lab> git commit -m "Update README for conflict feature"
[feature/conflict dbel5cf] Update README for conflict feature
...\\Lab\\Gitflow-Lab> git checkout develop
Switched to branch 'develop'
...\\Lab\\Gitflow-Lab> git branch
  feature/conflict
  feature/login
* develop

```

Screenshot 39 — Step 39

Step 40. Create the Develop Version

What you see: README.md shows "Gitflow Lab 5 - Develop Version".

Simple explanation: Now a different README version is prepared on develop.

```

EXPLORER
  Gitflow-Lab
    app.py
    README.md
  README.md
1  # Gitflow Lab - Develop Version

PROBLEMS OUTPUT DEBUG CONSOLETERMINAL PORTS
PS ..\\Lab\\Gitflow-Lab> git branch
* develop
  feature/conflict
  feature/login

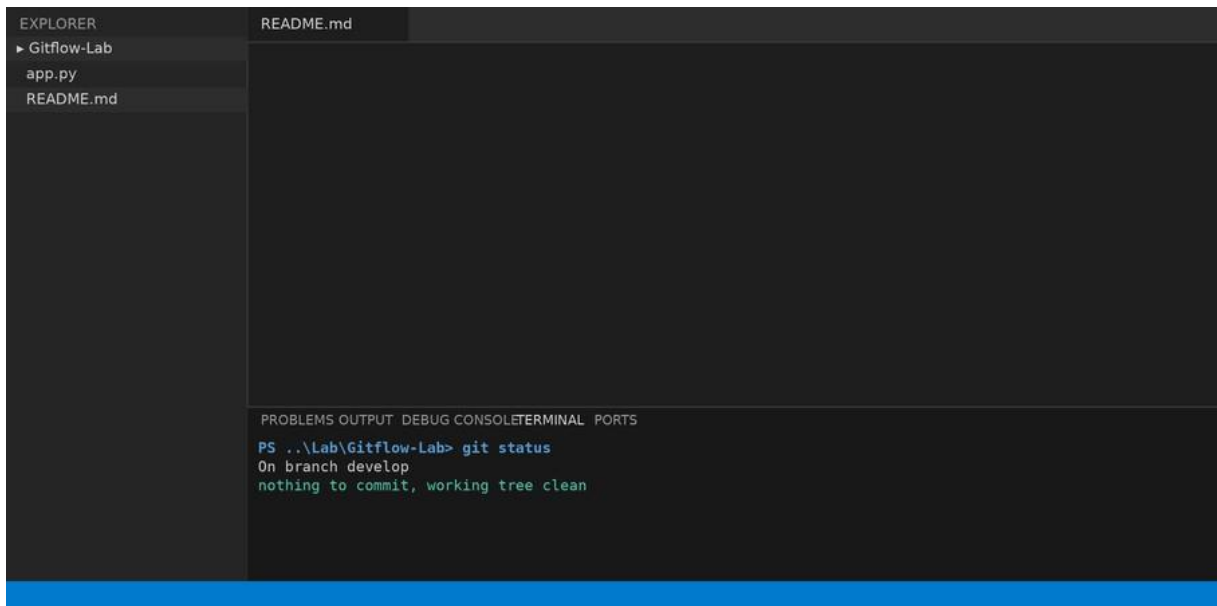
```

Screenshot 40 — Step 40

Step 41. Review the Branch History

What you see: VS Code Source Control/Timeline shows develop, feature/conflict, feature/login, and main in the history.

Simple explanation: The history helps us see that different branches contain different changes.

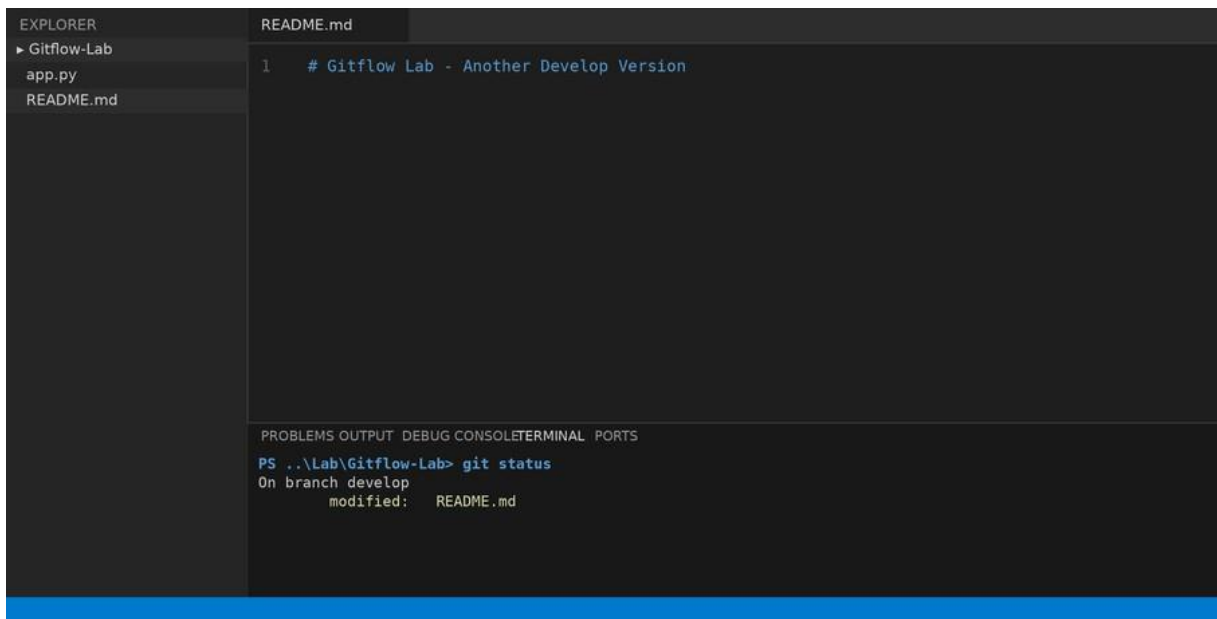


Screenshot 41 — Step 41

Step 42. Review the Project History

What you see: VS Code shows the Gitflow-Lab-5 project and the recorded README changes.

Simple explanation: The Timeline is useful for seeing saved file changes over time.

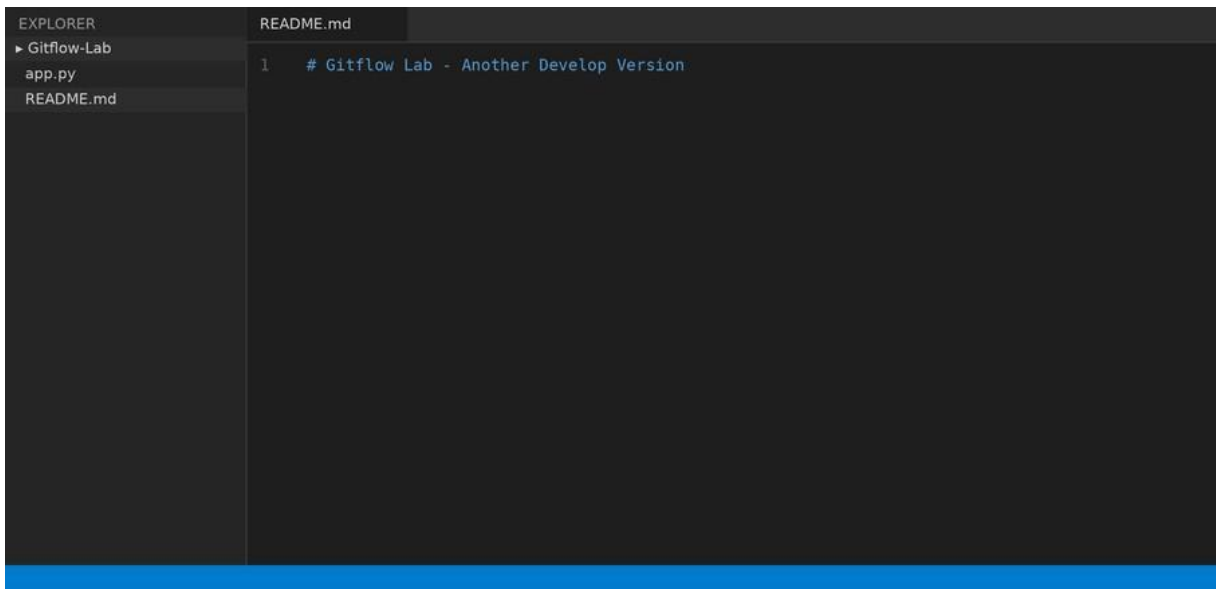


Screenshot 42 — Step 42

Step 43. Continue the Develop Change

What you see: README.md again shows the develop version in VS Code.

Simple explanation: This is the develop-side README content that will be different from the feature version.



Screenshot 43 — Step 43

Step 44. Check develop Status

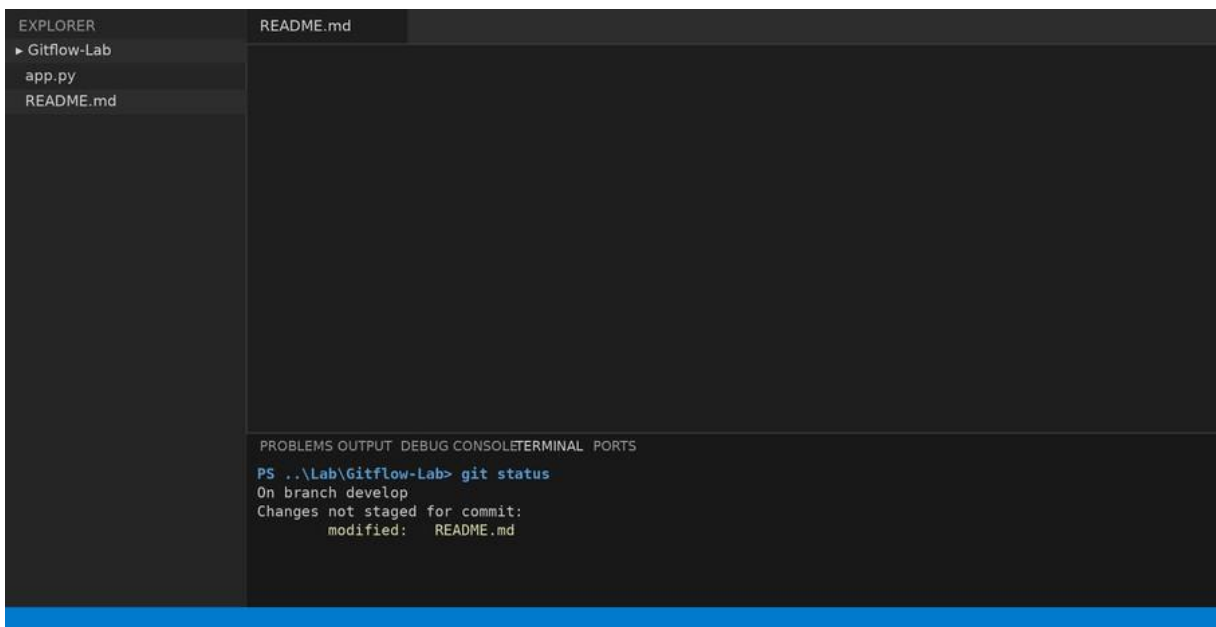
What you see: git status shows README.md modified on develop.

Simple explanation: Git has detected the change and it is ready to be staged.

Command:

```
git status
```

Purpose: Shows that README.md is modified.

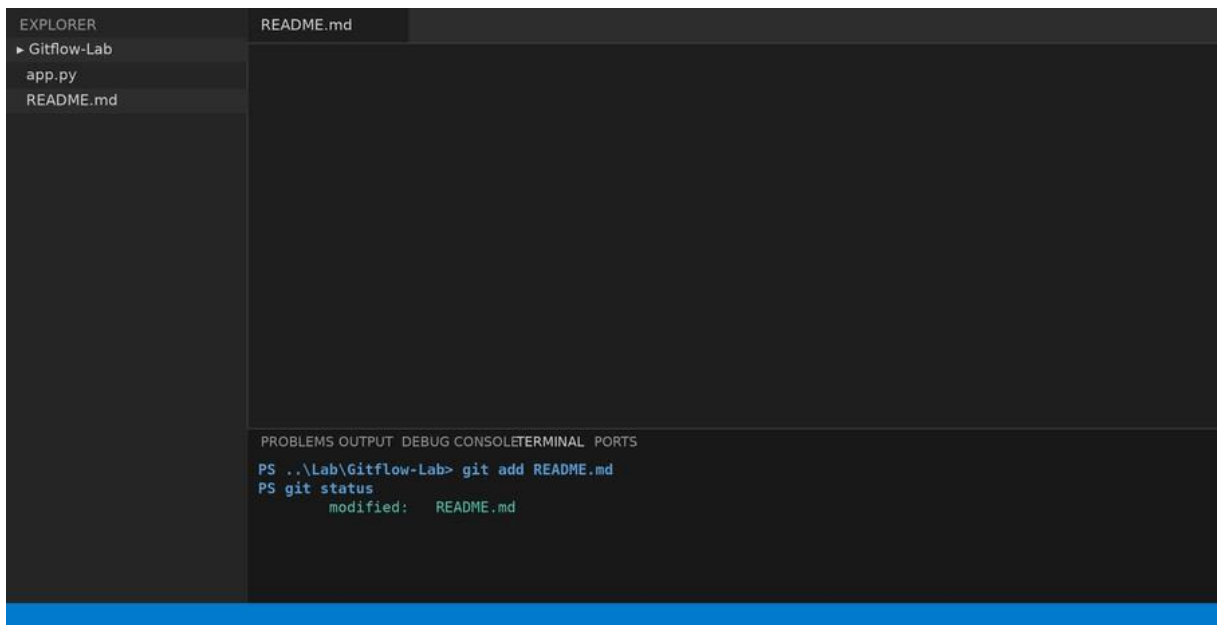


Screenshot 44 — Step 44

Step 45. Stage the Develop Change

What you see: VS Code Source Control shows the README.md change staged.

Simple explanation: The develop-side change is now ready for commit.



Screenshot 45 — Step 45

Step 46. Commit the Develop README

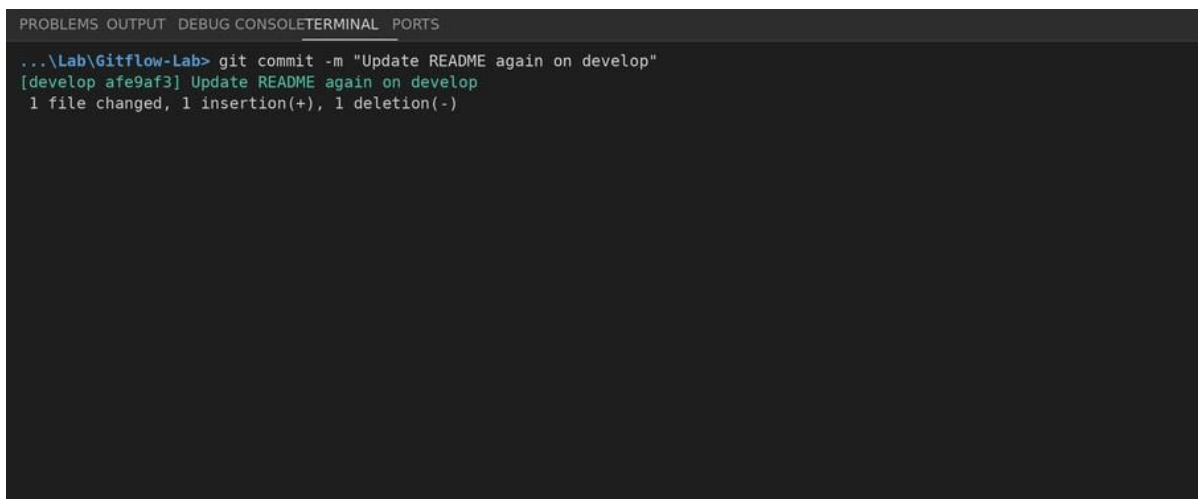
What you see: The terminal shows git add README.md, git status, and the commit command.

Simple explanation: This saves the develop-side README change in Git history.

Command:

```
git commit -m "Update README on develop"
```

Purpose: Saves the develop-side README change.

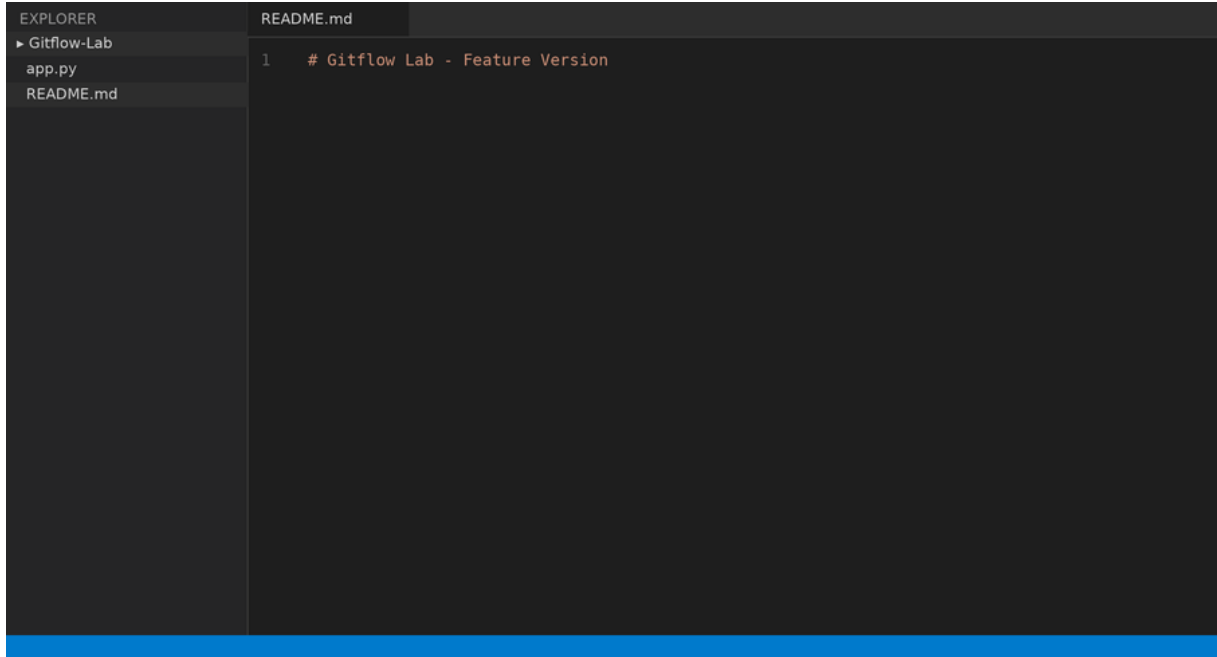


Screenshot 46 — Step 46

Step 47. Create the Conflict Version

What you see: README.md shows "Gitflow Lab 5 - Conflict Version".

Simple explanation: This is another visible version of the same README file.

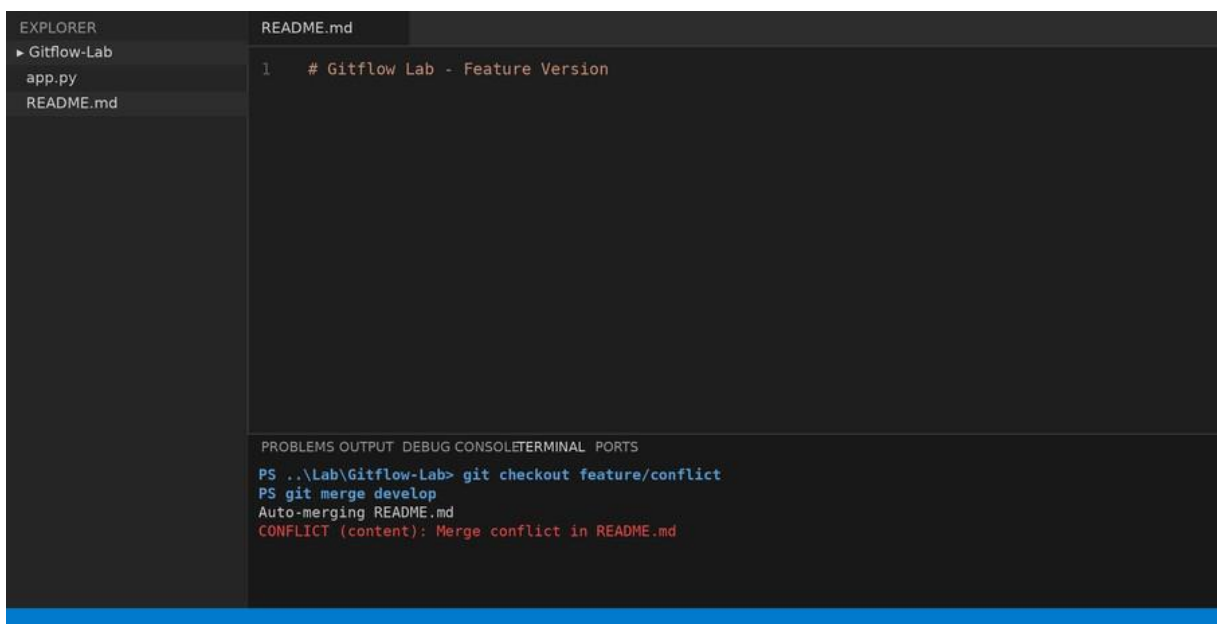


Screenshot 47 — Step 47

Step 48. Open the Merge State

What you see: VS Code shows README.md in the conflict workflow and Source Control information.

Simple explanation: The project is now ready for the merge that will produce the conflict.

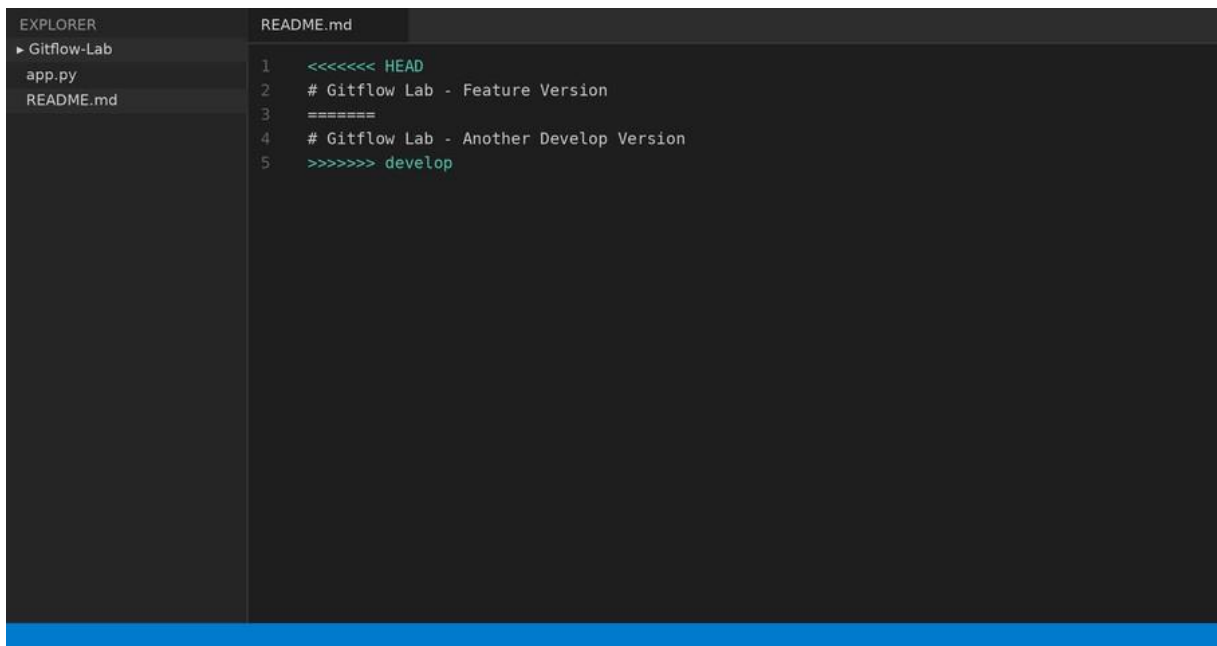


Screenshot 48 — Step 48

Step 49. Review the Conflict-Feature History

What you see: The VS Code Timeline shows the conflict-feature README update and saved files.

Simple explanation: The Timeline records the previous feature changes.



The screenshot shows the VS Code interface with the Explorer sidebar on the left and the README.md file open in the editor. The Explorer sidebar shows the project structure: Gitflow-Lab, app.py, and README.md. The README.md file content is as follows:

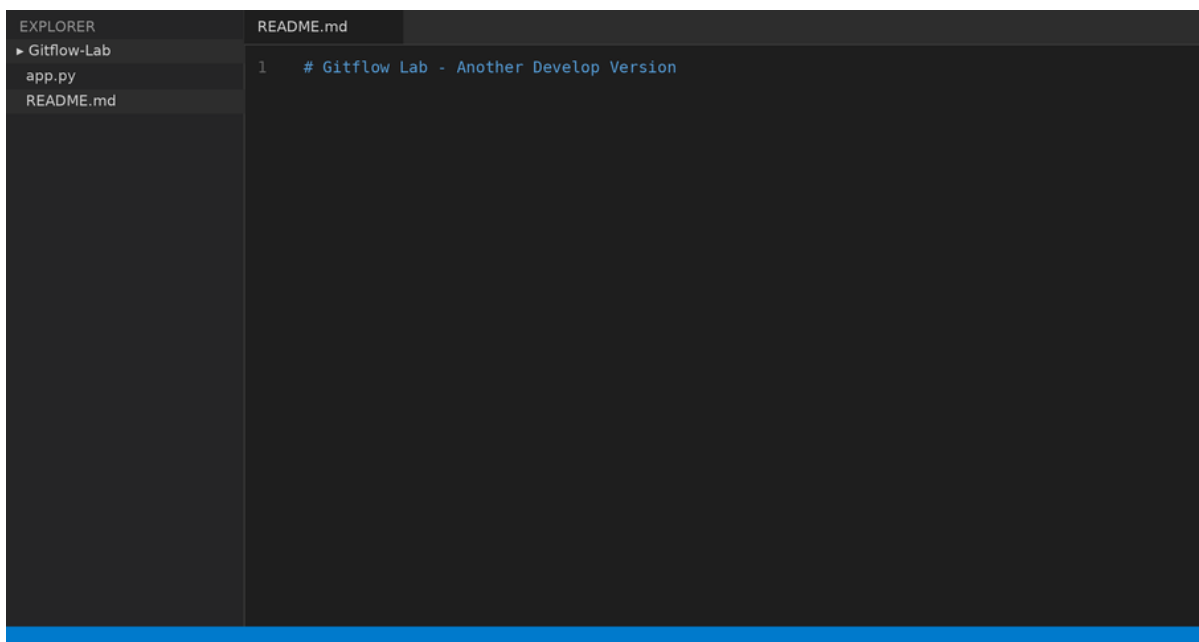
```
1 <<<<<< HEAD
2 # Gitflow Lab - Feature Version
3 =====
4 # Gitflow Lab - Another Develop Version
5 >>>>>> develop
```

Screenshot 49 — Step 49

Step 50. Review the Develop History

What you see: The VS Code Timeline shows the develop README update and saved files.

Simple explanation: The Timeline records the develop-side changes.



The screenshot shows the VS Code interface with the Explorer sidebar on the left and the README.md file open in the editor. The Explorer sidebar shows the project structure: Gitflow-Lab, app.py, and README.md. The README.md file content is as follows:

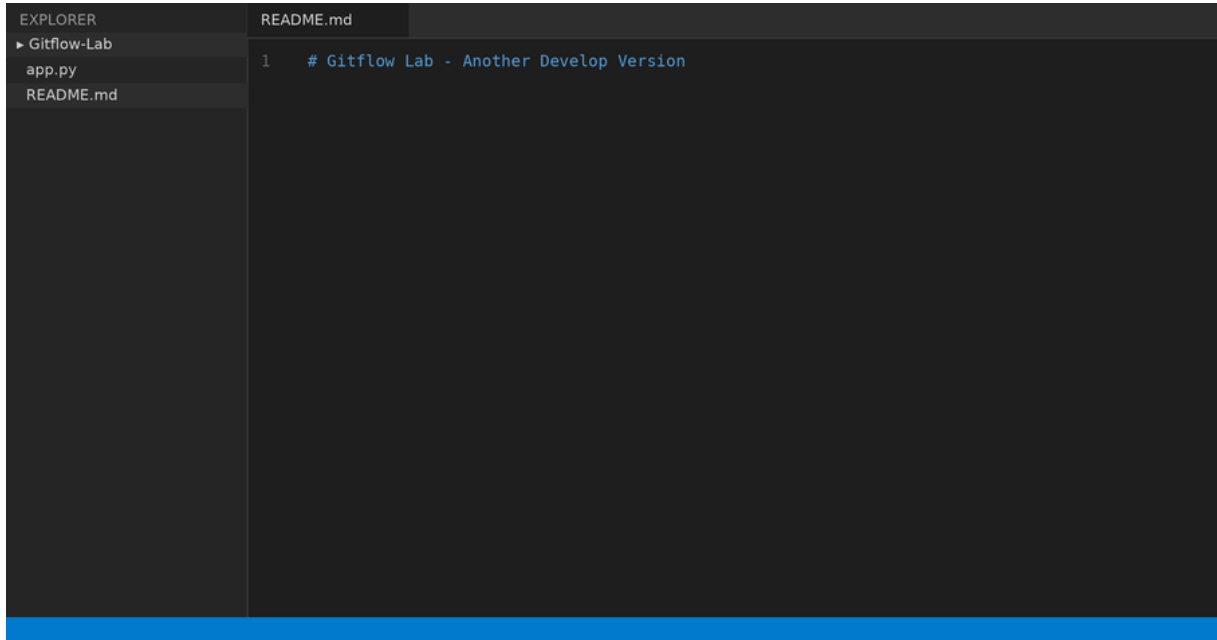
```
1 # Gitflow Lab - Another Develop Version
```

Screenshot 50 — Step 50

Step 51. Create Another Develop Version

What you see: README.md shows "Gitflow Lab 5 - Another Develop Version".

Simple explanation: The develop branch now has a version that is different from the feature branch version.

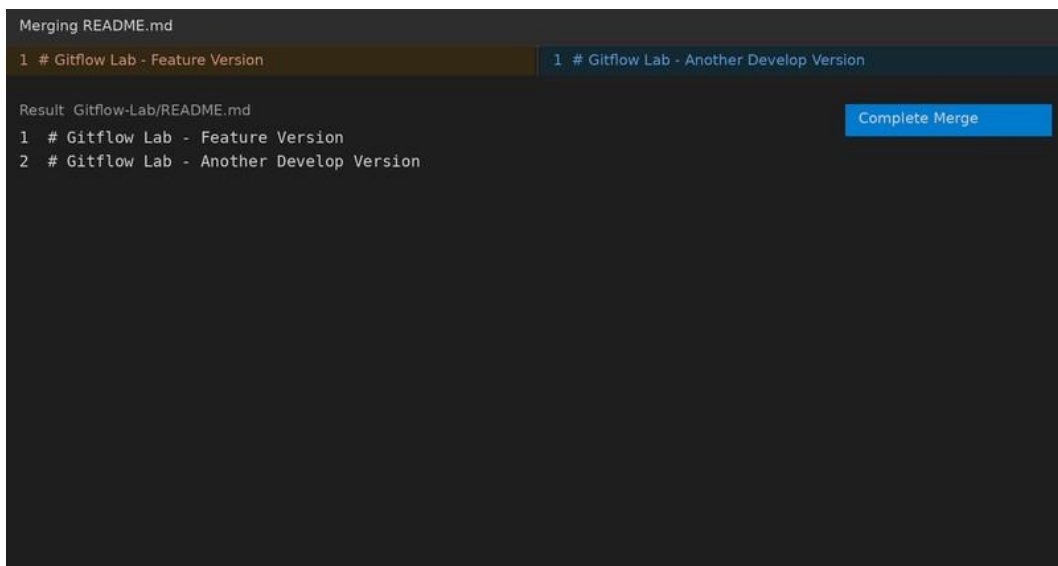


Screenshot 51 — Step 51

Step 52. Open the Merge Editor

What you see: VS Code Merge Editor shows Incoming develop content and Current feature/conflict content for README.md.

Simple explanation: Git cannot automatically decide which version to keep. The Merge Editor shows both sides.

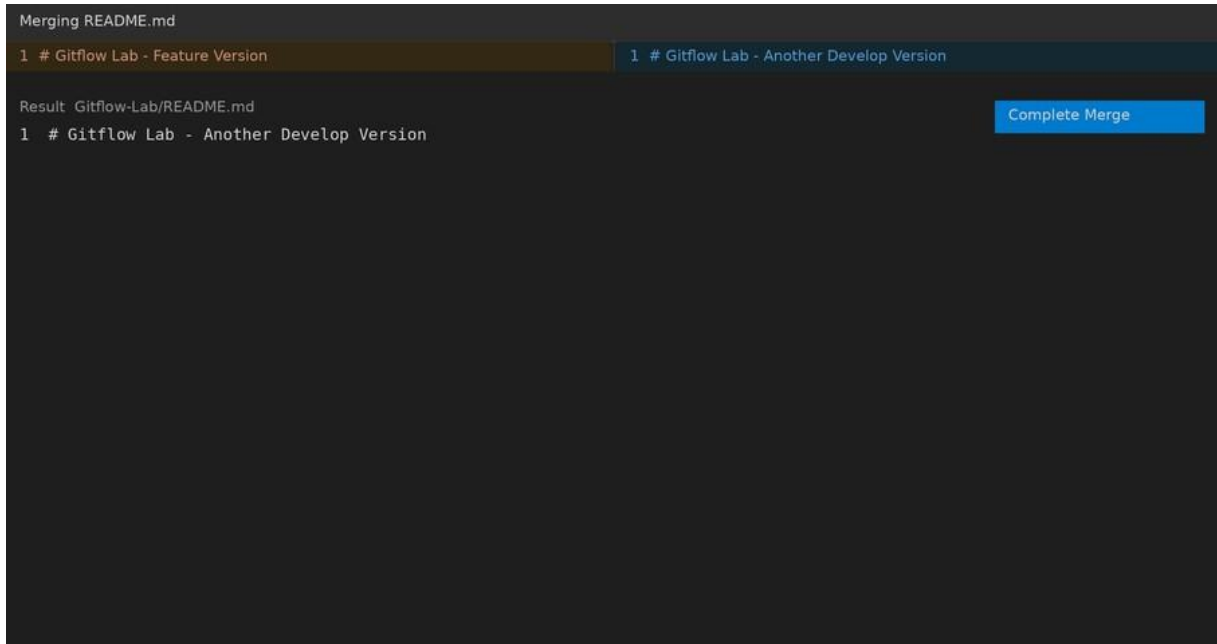


Screenshot 52 — Step 52

Step 53. See Both Versions Together

What you see: The Merge Editor shows the two README versions and the result area.

Simple explanation: Choose the correct content and make one final clean version in the result area.



Screenshot 53 — Step 53

Step 54. Complete the Conflict Resolution in Source Control

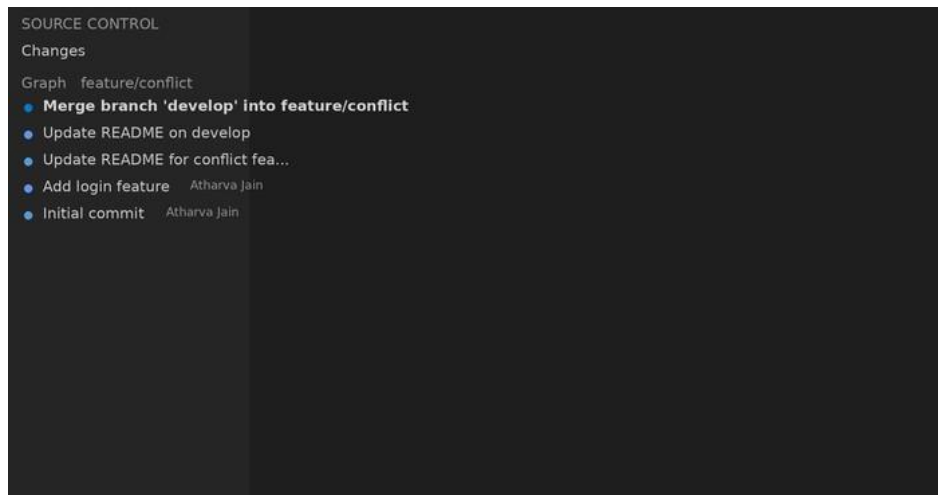
What you see: VS Code Source Control shows the merge state and the staged README.md.

Simple explanation: After the file is corrected, it must be staged and the merge must be completed with a commit.

Command:

```
git add README.md
```

Purpose: Stages the resolved README file.

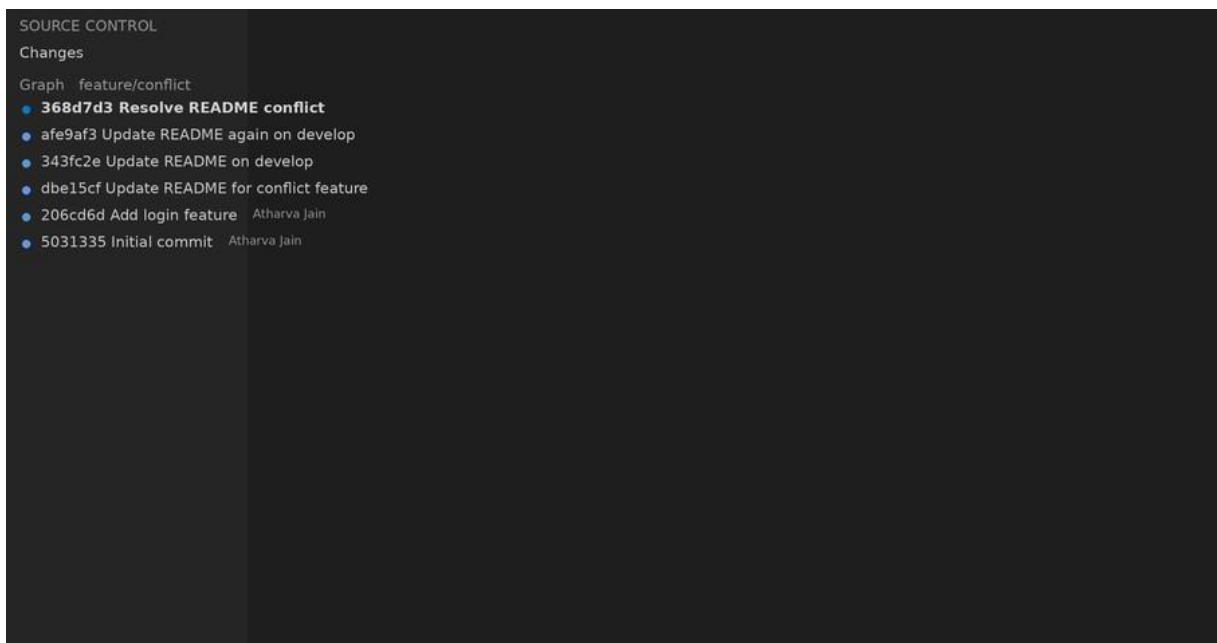


Screenshot 54 — Step 54

Step 55. Review the Merge History

What you see: The terminal and Source Control show the merge history and Fast-forward/app.py information.

Simple explanation: This is a history check after the merge work.

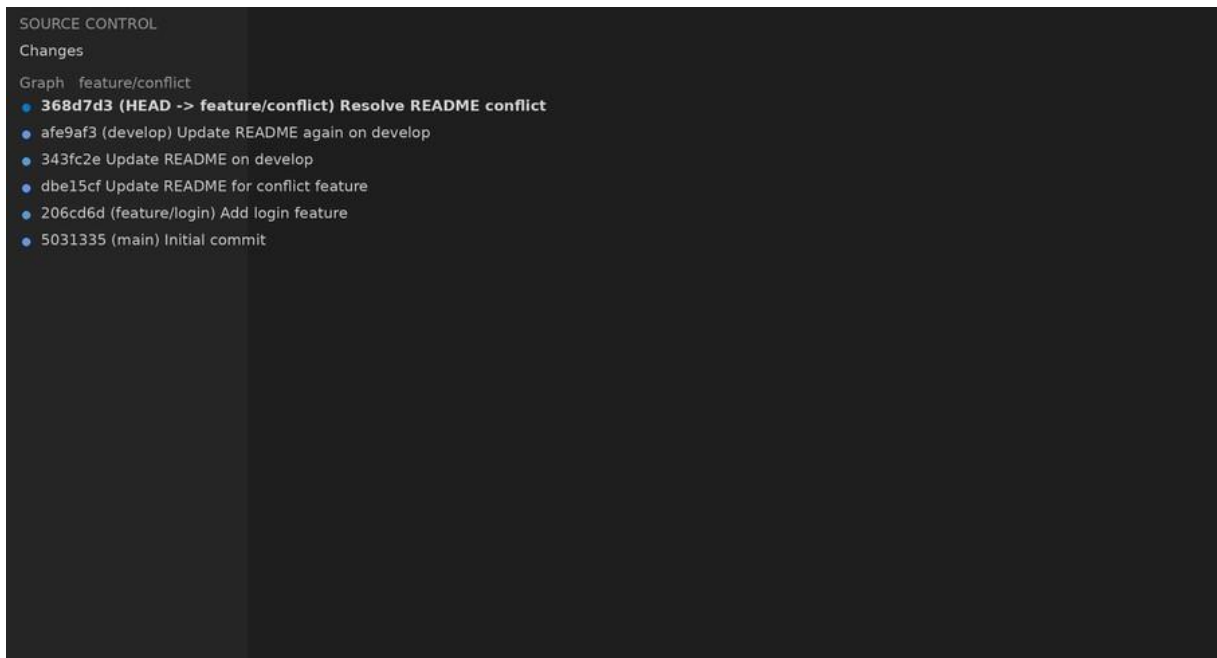


Screenshot 55 — Step 55

Step 56. Review the Final Local History

What you see: VS Code shows the project history after the conflict workflow.

Simple explanation: The history should now contain the work from the branches and the merge.

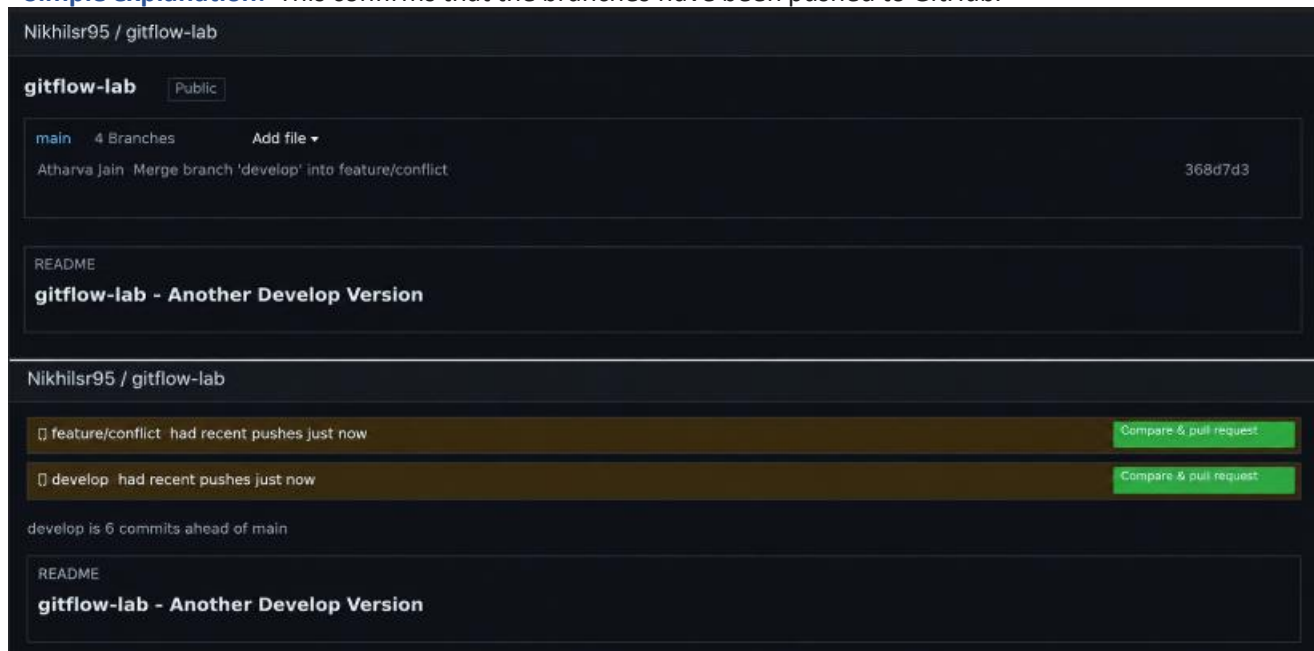


Screenshot 56 — Step 56

Step 57. Check GitHub Branches and Recent Pushes

What you see: GitHub shows recent pushes for feature/conflict and develop.

Simple explanation: This confirms that the branches have been pushed to GitHub.

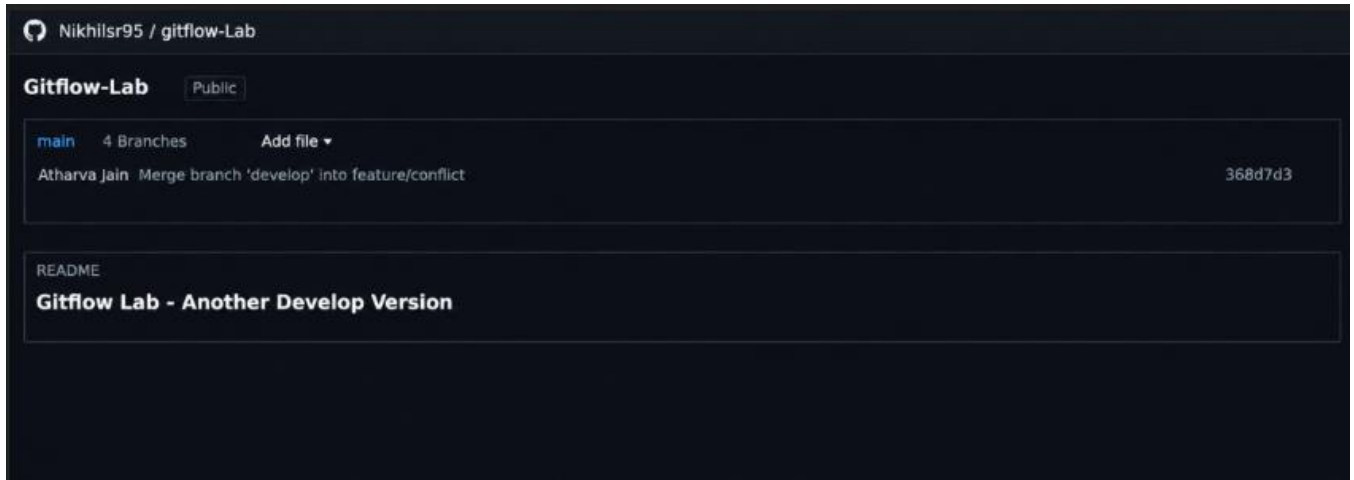


Screenshot 57 — Step 57

Step 58. Open the Final develop Branch

What you see: GitHub shows the develop branch with README.md, app.py, the commit history, and the README content.

Simple explanation: This is the final verification screen. The develop branch contains the integrated project work.



Screenshot 58 — Step 58

Final Revision

Remember the main flow: Create GitHub repository → connect the local project → create develop → create feature/login → commit → push → merge into develop → create feature/conflict → make different changes → merge → resolve the conflict → stage → commit → push → verify on GitHub.

Most Important Commands

- `git status` —
- `git branch` —
- `git checkout -b branch-name` —
- `git add file-name` —
- `git commit -m "message"` —
- `git push` —
- `git merge branch-name` —
- `python app.py` —